

A Unified Executors Proposal for C++ | P0443R13

Title: A Unified Executors Proposal for C++

Authors: Jared Hoberock, jhoberock@nvidia.com
Michael Garland, mgarland@nvidia.com
Chris Kohlhoff, chris@kohlhoff.com
Chris Mysisen, mysen@google.com
Carter Edwards, hcedwar@sandia.gov
Gordon Brown, gordon@codeplay.com
David Hollman, dshollm@sandia.gov
Lee Howes, lwh@fb.com
Kirk Shoop, kirkshoop@fb.com
Lewis Baker, lbaker@fb.com
Eric Niebler, eniebler@fb.com

Other Contributors: Hans Boehm, hboehm@google.com
Thomas Heller, thom.heller@gmail.com
Bryce Leibel, bryceleibel@gmail.com
Hartmut Kaiser, hartmut.kaiser@gmail.com
Bryce Leibel, bryceleibel@gmail.com
Gor Nishanov, gorn@microsoft.com
Thomas Rodgers, rodgert@twrogers.com
Michael Wong, michael@codeplay.com

Document Number: P0443R13

Date: 2020-03-02

Audience: SG1 - Concurrency and Parallelism, LEWG

Reply-to: sg1-exec@googlegroups.com

Abstract: This paper proposes [a programming model](#) for executors, which are modular components for creating execution, and senders, which are lazy descriptions of execution.

1 Design Document

1.1 Motivation

When we imagine the future of C++ programs, we envision elegant compositions of networked, asynchronous parallel computations accelerated by diverse hardware, ranging from tiny mobile devices to giant supercomputers. In the present, hardware diversity is greater than ever, but C++ programmers lack satisfying parallel programming tools for them. Industrial-strength concurrency primitives like `std::thread` and `std::atomic` are powerful but hazardous. `std::async` and `std::future` suffer from well-known problems. And the standard algorithms library, though parallelized, remains inflexible and non-composable.

To address these temporary challenges and build toward the future, C++ must lay a foundation for controlling program execution. First, C++ **must provide flexible facilities to control where and when work happens**. This paper proposes a design for those facilities. After [much discussion and collaboration](#), SG1 adopted this design by universal consensus at the Cologne meeting in 2019.

1.2 Usage Example

This proposal defines requirements for two key components of execution: a work execution interface and a representation of work and their interrelationships. Respectively, these are **executors** and **senders and receivers**:

```
// make P0443 APIs in namespace std::execution available
using namespace std::execution;

// get an executor from somewhere, e.g. a thread pool
std::static_thread_pool pool(16);
executor auto ex = pool.executor();
```

```
// use the executor to describe where some high-level library should execute its work
perform_business_logic(ex);

// alternatively, use primitive P0443 APIs directly

// immediately submit work to the pool
execute(ex, []{ std::cout << "Hello world from the thread pool!"; });

// immediately submit work to the pool and require this thread to block until completion
execute(std::require(ex, blocking.always), foo);

// describe a chain of dependent work to submit later
sender auto begin      = schedule(ex);
sender auto hi_again    = then(begin, []{ std::cout << "Hi again! Have an int."; return 13; });
sender auto work        = then(hi_again, [](int arg) { return arg + 42; });

// prints the final result
receiver auto print_result = as_receiver([](int arg) { std::cout << "Received " << std::endl; });

// submit the work for execution on the pool by combining with the receiver
submit(work, print_result);

// Blue: proposed by P0443. Teal: possible extensions.
```

1.3 Executors Execute Work

As lightweight handles, executors impose uniform access to execution contexts.

Executors provide a uniform interface for work creation by abstracting underlying resources where work physically executes. The previous code example’s underlying resource was a thread pool. Other examples include SIMD units, GPU runtimes, or simply the current thread. In general, we call such resources **execution contexts**. As lightweight handles, executors impose uniform access to execution contexts. Uniformity enables control over where work executes, even when it is executed indirectly behind library interfaces.

The basic executor interface is the execute function through which clients execute work:

```
// obtain an executor
executor auto ex = ...

// define our work as a nullary invocable
invocable auto work = []{ cout << "My work" << endl; };

// execute our work via the execute customization point
execute(ex, work);
```

On its own, execute is a primitive “fire-and-forget”-style interface. It accepts a single nullary invocable, and returns nothing to identify or interact with the work it creates. In this way, it trades convenience for universality. As a consequence, we expect most programmers to interact with executors via more convenient higher-level libraries, our envisioned asynchronous STL being such an example.

Consider how std::async could be extended to interoperate with executors enabling client control over execution:

```
template<class Executor, class F, class Args...>
future<invoke_result_t<F,Args...>> async(const Executor& ex, F&& f, Args&&... args) {
    // package up the work
    packaged_task work(forward<F>(f), forward<Args>(args)...);

    // get the future
    auto result = work.get_future();

    // execute work on the given executor
    execution::execute(ex, move(work));

    return result;
}
```

The benefit of such an extension is that a client can select from among multiple thread pools to control exactly which pool std::async uses simply by providing a corresponding executor. Inconveniences of work packaging and submission become the library’s responsibility.

Authoring executors. Programmers author custom executor types by defining a type with an execute function. Consider the implementation of an executor whose execute function executes the client’s work “inline”:

```
struct inline_executor {
```

```
// define execute
template<class F>
void execute(F&& f) const noexcept {
    std::invoke(std::forward<F>(f));
}

// enable comparisons
auto operator<=>(const inline_executor&) const = default;
};
```

Additionally, a comparison function determines whether two executor objects refer to the same underlying resource and therefore execute with equivalent semantics. Concepts `executor` and `executor_of` summarize these requirements. The former validates executors in isolation; the latter, when both executor and work are available.

Executor customization can accelerate execution or introduce novel behavior. The previous example demonstrated custom execution at the granularity of a new executor type, but finer-grained and coarser-grained customization techniques are also possible. These are **executor properties** and **control structures**, respectively.

Executor properties communicate optional behavioral requirements beyond the minimal contract of `execute`, and this proposal specifies several. We expect expert implementors to impose these requirements beneath higher-level abstractions. In principle, optional, dynamic data members or function parameters could communicate these requirements, but C++ requires the ability to introduce customization at compile time. Moreover, optional parameters lead to [combinatorially many function variants](#).

Instead, statically-actionable properties factor such requirements and thereby avoid a combinatorial explosion of executor APIs. For example, consider the requirement to execute blocking work with priority. An unscalable design might embed these options into the `execute` interface by multiplying individual factors into separate functions: `execute`, `blocking_execute`, `execute_with_priority`, `blocking_execute_with_priority`, etc.

Executors avoid this unscalable situation by adopting [P1393](#)'s properties design based on `require` and `prefer`:

```
// obtain an executor
executor auto ex = ...;

// require the execute operation to block
executor auto blocking_ex = std::require(ex, execution::blocking.always);

// prefer to execute with a particular priority p
executor auto blocking_ex_with_priority = std::prefer(blocking_ex, execution::priority(p));

// execute my blocking, possibly prioritized work
execution::execute(blocking_ex_with_priority, work);
```

Each application of `require` or `prefer` transforms an executor into one with the requested property. In this example, if `ex` cannot be transformed into a blocking executor, the call to `require` will fail to compile. `prefer` is a weaker request used to communicate hints and consequently always succeeds because it may ignore the request.

Consider a version of `std::async` which *never* blocks the caller:

```
template<executor E, class F, class... Args>
auto really_async(const E& ex, F&& f, Args&&... args) {
    using namespace execution;

    // package up the work
    packaged_task work(forward<F>(f), forward<Args>(args)...);

    // get the future
    auto result = work.get_future();

    // execute the nonblocking work on the given executor
    execute(require(ex, blocking.never), move(work));

    return result;
}
```

Such an enhancement could address a well-known hazard of `std::async`:

```
// confusingly, always blocks in the returned but discarded future's destructor
std::async(foo);

// *never* blocks
really_async(foo);
```

Control structures permit customizations at a higher level of abstraction by allowing executors to “hook” them and is useful when an efficient implementation is possible on a particular execution context. The first such control structure this proposal

defines `bulk_execute`, which creates a group of function invocations in a single operation. This pattern permits a wide range of efficient implementations and is of fundamental importance to C++ programs and the standard library.

By default, `bulk_execute` invokes `execute` repeatedly, but repeatedly executing individual work items is inefficient at scale. Consequently, many platforms provide APIs that explicitly and efficiently execute bulk work. In such cases, a custom `bulk_execute` avoids inefficient platform interactions via direct access to these accelerated bulk APIs while also optimizing the use of scalar APIs.

`bulk_execute` receives an invocable and an invocation count. Consider a possible implementation:

```
struct simd_executor : inline_executor { // first, satisfy executor requirements via inheritance
    template<class F>
    simd_sender bulk_execute(F f, size_t n) const {
        #pragma simd
        for(size_t i = 0; i != n; ++i) {
            std::invoke(f, i);
        }

        return {};
    }
};
```

To accelerate `bulk_execute`, `simd_executor` uses a SIMD loop.

`bulk_execute` should be used in cases where multiple pieces of work are available at once:

```
template<class Executor, class F, class Range>
void my_for_each(const Executor& ex, F f, Range rng) {
    // request bulk execution, receive a sender
    sender auto s = execution::bulk_execute(ex, [=](size_t i) {
        f(rng[i]);
    }, std::ranges::size(rng));

    // initiate execution and wait for it to complete
    execution::sync_wait(s);
}
```

`simd_executor`'s particular `bulk_execute` implementation executes “eagerly”, but `bulk_execute`'s semantics do not require it. As `my_for_each` demonstrates, unlike `execute`, `bulk_execute` is an example of a “lazy” operation whose execution may be optionally postponed. The token this `bulk_execute` returns is an example of a sender a client may use to initiate execution or otherwise interact with the work. For example, calling `sync_wait` on the sender ensures that the bulk work completes before the caller continues. Senders and receivers are the subject of the next section.

1.4 Senders and Receivers Represent Work

The executor concept addresses a basic need of executing a single operation in a specified execution context. The expressive power of executor is limited, however: since `execute` returns `void` instead of a handle to the just-scheduled work, the executor abstraction gives no generic way to chain operations and thereby propagate values, errors, and cancellation signals downstream; no way to handle scheduling errors occurring between when work submission and execution; and no convenient way to control the allocation and lifetime of state associated with an operation.

Without such controls, it is not possible to define Generic (in the Stepanov sense) asynchronous algorithms that compose efficiently with sensible default implementations. To fill this gap, this paper proposes two related abstractions, sender and receiver, concretely motivated below.

1.4.1 Generic async algorithm example: `retry`

`retry` is the kind of Generic algorithm senders and receivers enable. It has simple semantics: schedule work on an execution context; if the execution succeeds, done; otherwise, if the user requests cancellation, done; otherwise, if a scheduling error occurs, try again.

```
template<invocable Fn>
void retry(executor_of<Fn> auto ex, Fn fn) {
    // ???
}
```

Executors alone prohibit a generic implementation because they lack a portable way to intercept and react to scheduling errors. Later we show how this algorithm might look when implemented with senders and receivers.

1.4.2 Goal: an asynchronous STL

Suitably chosen concepts driving the definition of Generic async algorithms like `retry` streamline the creation of efficient, asynchronous graphs of work. Here is some sample syntax for the sorts of async programs we envision (borrowed from [P1897](#)):

```
sender auto s = just(3) |                                // produce '3' immediately
    via(scheduler1) |                                     // transition context
    then([](int a){return a+1;}) |                       // chain continuation
    then([](int a){return a*2;}) |                       // chain another continuation
    via(scheduler2) |                                     // transition context
    handle_error([](auto e){return just(3);});           // with default value on errors
int r = sync_wait(s);                                     // wait for the result
```

It should be possible to replace `just(3)` with a call to any asynchronous API whose return type satisfies the correct concept and maintain this program’s correctness. Generic algorithms like `when_all` and `when_any` would permit users to express fork/join concurrency in their DAGs. As with STL’s iterator abstraction, the cost of satisfying the conceptual requirements are offset by the expressivity of a large reusable and composable library of algorithms.

1.4.3 Current techniques

There are many techniques for creating chains of dependent asynchronous execution. Ordinary callbacks have enjoyed success in C++ and elsewhere for years. Modern codebases have switched to variations of future abstractions that support continuations (e.g., `std::experimental::future::then`). In C++20 and beyond, we could imagine standardizing on coroutines, so that launching an async operation returns an awaitable. Each of these approaches has strengths and weaknesses.

Futures, as traditionally realized, require the dynamic allocation and management of a shared state, synchronization, and typically type-erasure of work and continuation. Many of these costs are inherent in the nature of “future” as a handle to an operation that is already scheduled for execution. These expenses rule out the future abstraction for many uses and makes it a poor choice for a basis of a Generic mechanism.

Coroutines suffer many of the same problems but can avoid synchronizing when chaining dependent work because they typically start suspended. In many cases, coroutine frames require unavoidable dynamic allocation. Consequently, coroutines in embedded or heterogeneous environments require great attention to detail. Neither are coroutines good candidates for cancellation because the early and safe termination of coordinating coroutines requires unsatisfying solutions. On the one hand, exceptions are inefficient and disallowed in many environments. Alternatively, clumsy *ad hoc* mechanisms, whereby `co_yield` returns a status code, hinder correctness. [P1662](#) provides a complete discussion.

Callbacks are the simplest, most powerful, and most efficient mechanism for creating chains of work, but suffer problems of their own. Callbacks must propagate either errors or values. This simple requirement yields many different interface possibilities, but the lack of a standard obstructs Generic design. Additionally, few of these possibilities accomodate cancellation signals when the user requests upstream work to stop and clean up.

1.5 Receiver, sender, and scheduler

With the preceding as motivation, we introduce primitives to address the needs of Generic asynchronous programming in the presence of value, error, and cancellation propagation.

1.5.1 Receiver

A *receiver* is simply a callback with a particular interface and semantics. Unlike a traditional callback which uses function-call syntax and a single signature handling both success and error cases, a receiver has three separate channels for value, error, and “done” (aka cancelled).

These channels are specified as customization points, and a type `R` modeling `receiver_of<R,Ts...>` supports them:

```
std::execution::set_value(r, ts...); // signal success, but set_value itself may fail
std::execution::set_error(r, ep);    // signal error (ep is std::exception_ptr), never fails
std::execution::set_done(r);         // signal stopped, never fails
```

Exactly one of the three functions must be called on a receiver before it is destroyed. Each of these interfaces is considered “terminal”. That is, a particular receiver may assume that if one is called, no others ever will be. The one exception being if `set_value` exits with an exception, the receiver is not yet complete. Consequently, another function must be called on it before it is destroyed. After a failed call to `set_value`, correctness requires a subsequent call either to `set_error` or `set_done`; a receiver need not guarantee that a second call to `set_value` is well-formed. Collectively, these requirements are the “*receiver contract*”.

Although receiver’s interface appears novel at first glance, it remains just a callback. Moreover, receiver’s novelty disappears when recognizing that `std::promise`’s `set_value` and `set_exception` provide essentially the same interface. This choice of interface and semantics, along with sender, facilitate the Generic implementation of many useful async algorithms like `retry`.

1.5.2 Sender

A sender represents work that has not been scheduled for execution yet, to which one must add a continuation (a receiver) and then “launch”, or enqueue for execution. A sender’s duty to its connected receiver is to fulfill the *receiver contract* by ensuring that one of the three receiver functions returns normally.

Earlier versions of this paper fused these two operations — attach a continuation and launch for execution — into the single operation `submit`. This paper proposes to split `submit` into a `connect` step that packages a sender and a receiver into an operation state, and a `start` step that logically starts the operation and schedules the receiver completion-signalling methods to be called when the operation completes.

```
// P0443R12
std::execution::submit(snd, rec);

// P0443R13
auto state = std::execution::connect(snd, rec);
// ... later
std::execution::start(state);
```

This split offers interesting opportunities for optimization, and [harmonizes senders with coroutines](#).

The sender concept itself places no requirements on the execution context on which a sender’s work executes. Instead, specific models of the sender concept may offer stronger guarantees about the context from which the receiver’s methods will be invoked. This is particularly true of the senders created by a scheduler.

1.5.3 Scheduler

Many generic async algorithms create multiple execution agents on the same execution context. Therefore, it is insufficient to parameterize these algorithms with a single-shot sender completing in a known context. Rather, it makes sense to pass these algorithms a factory of single-shot senders. Such a factory is called a “scheduler”, and it has a single basis operation: `schedule`:

```
sender auto s = std::execution::schedule(sched);
// OK, s is a single-shot sender of void that completes in sched's execution context
```

Like executors, schedulers act as handles to an execution context. Unlike executors, schedulers submit execution lazily, but a single type may simultaneously model both concepts. We envision that subsumptions of the `scheduler` concept will add the ability to postpone or cancel execution until after some time period has elapsed.

1.6 Senders, receivers, and generic algorithms

Useful concepts constrain generic algorithms while allowing default implementations via those concepts’ basis operations. Below, we show how these sender and receiver provide efficient default implementations of common async algorithms. We envision that most generic async algorithms will be implemented as taking a sender and returning a sender whose `connect` method wraps its receiver an adaptor that implements the algorithm’s logic. The `then` algorithm below, which chains a continuation function on a sender, is a simple demonstration.

1.6.1 Algorithm `then`

The following code implements a `then` algorithm that, like `std::experimental::future::then`, schedules a function to be applied to the result of an asynchronous operation when available. This code demonstrates how an algorithm can adapt receivers to codify the algorithm’s logic.

```
template<receiver R, class F>
struct _then_receiver : R { // for exposition, inherit set_error and set_done from R
    F f_;

    // Customize set_value by invoking the callable and passing the result to the base class
    template<class... As>
        requires receiver_of<R, invoke_result_t<F, As...>>
    void set_value(Args&&... args) && noexcept(/*...*/) {
        ::set_value((R&&) *this, invoke((F&&) f_, (As&&) as...));
    }

    // Not shown: handle the case when the callable returns void
};

template<sender S, class F>
struct _then_sender : _sender_base {
    S s_;
    F f_;
```



```

template<receiver R>
    requires sender_to<S, _then_receiver<R, F>>
    state_t<S, _then_receiver<R, F>> connect(R r) && {
        return ::connect((S&&)s_, _then_receiver<R, F>{(R&&)r, (F&&)f_});
    }
};

template<sender S, class F>
sender auto then(S s, F f) {
    return _then_sender{{}, (S&&)s, (F&&)f};
}

```

Given some asynchronous, sender-returning API `async_foo`, a user of `then` can execute some code once the async result is available:

```
sender auto s = then(async_foo(args...), [](auto result) { /* stuff... */ });
```

This builds a composed asynchronous operation. When the user wants to schedule this operation for execution, they would connect a receiver, and then call `start` on the resulting operation state.

Scheduling work on an execution context can also be done with `then`. Given a `static_thread_pool` object `pool` that satisfied the scheduler concept, a user may do the following:

```
sender auto s = then(
    std::execution::schedule( pool ),
    []{ std::printf("hello world"); } );
```

This creates a sender that, when submitted, will call `printf` from a thread in the thread pool.

There exist heterogeneous computing environments that are unable to execute arbitrary code. For those, an implementation of `then` as shown above would either not work or would incur the cost of a transition to the host in order to execute the unknown code. Therefore, `then` itself and several other fundamental algorithmic primitives, would themselves need to be customizable on a per-execution context basis.

A full working example of `then` can be found here: <https://godbolt.org/z/dafqM->

1.6.2 Algorithm `retry`

As described above, the idea of `retry` is to retry the async operation on failure, but not on success or cancellation. Key to a correct generic implementation of `retry` is the ability to distinguish the error case from the cancelled case.

As with the `then` algorithm, the `retry` algorithm places the logic of the algorithm into a custom receiver to which the sender to be retried is connect-ed. That custom receiver has `set_value` and `set_done` members that simply pass their signals through unmodified. The `set_error` member, on the other hand, reconstructs the operation state in-place by making another call to `connect` with the original sender and a new instance of the custom receiver. That new operation state is then `start`-ed again, which effectively causes the original sender to be retried.

[The appendix](#) lists the source of the `retry` algorithm. Note that the signature of the `retry` algorithm is simply:

```
sender auto retry(sender auto s);
```

That is, it is not parameterized on an execution context on which to retry the operation. That is because we can assume the existence of a function `on` which schedules a sender for execution on a specified execution context:

```
sender auto on(sender auto s, scheduler auto sched);
```

Given these two functions, a user can simply do `retry(on(s, sched))` to retry an operation on a particular execution context.

1.6.3 Toward an asynchronous STL

The algorithms `then` and `retry` are only two of many interesting Generic asynchronous algorithms that are expressible in terms of senders and receivers. Two other important algorithms are `on` and `via`, the former which schedules a sender for execution on a particular scheduler, and the latter which causes a sender's *continuations* to be run on a particular scheduler. In this way, chains of asynchronous computation can be created that transition from one execution context to another.

Other important algorithms are `when_all` and `when_any`, encapsulating fork/join semantics. With these algorithms and others, entire DAGs of async computation can be created and executed. `when_any` can in turn be used to implement a generic `timeout` algorithm, together with a sender that sleeps for a duration and then sends a “done” signal, and so these algorithms compose.

In short, sender/receiver permits a rich set of Generic asynchronous algorithms to sit alongside Stepanov's sequence algorithms

in the STL. Asynchronous APIs that return senders would be usable with these Generic algorithms, increasing reusability. [P1897](#) suggest an initial set of these algorithms.

1.7 Summary

We envision a future when C++ programmers can express asynchronous, parallel execution of work on diverse hardware resources through elegant standard interfaces. This proposal provides a foundation for flexible execution and is our initial step towards that goal. **Executors** represent hardware resources that execute work. **Senders and receivers** represent lazily-constructed asynchronous DAGs of work. These primitives empower programmers to control where and when work happens.

2 Proposed Wording

2.1 Execution Support Library

2.1.1 General

This Clause describes components supporting execution of function objects [function.objects].

(The following definition appears in working draft N4762 [thread.req.lockable.general])

An *execution agent* is an entity such as a thread that may perform work in parallel with other execution agents. [Note: Implementations or users may introduce other kinds of agents such as processes or thread-pool tasks. –end note] The calling agent is determined by context; e.g., the calling thread that contains the call, and so on.

An execution agent invokes a function object within an *execution context* such as the calling thread or thread-pool. An *executor* submits a function object to an execution context to be invoked by an execution agent within that execution context. [Note: Invocation of the function object may be inlined such as when the execution context is the calling thread, or may be scheduled such as when the execution context is a thread-pool with task scheduler. –end note] An executor may submit a function object with *execution properties* that specify how the submission and invocation of the function object interacts with the submitting thread and execution context, including forward progress guarantees [intro.progress].

For the intent of this library and extensions to this library, the *lifetime of an execution agent* begins before the function object is invoked and ends after this invocation completes, either normally or having thrown an exception.

2.1.2 Header <execution> synopsis

```
namespace std {
namespace execution {

    // Exception types:

    extern runtime_error const invocation_error; // exposition only
    struct receiver_invocation_error : runtime_error, nested_exception {
        receiver_invocation_error() noexcept
            : runtime_error(invocation_error), nested_exception() {}
    };

    // Invocable archetype

    using invocable_archetype = unspecified;

    // Customization points:

    inline namespace unspecified{
        inline constexpr unspecified set_value = unspecified;

        inline constexpr unspecified set_done = unspecified;

        inline constexpr unspecified set_error = unspecified;

        inline constexpr unspecified execute = unspecified;

        inline constexpr unspecified connect = unspecified;

        inline constexpr unspecified start = unspecified;

        inline constexpr unspecified submit = unspecified;

        inline constexpr unspecified schedule = unspecified;
```



```

    inline constexpr unspecified bulk_execute = unspecified;
}

template<class S, class R>
    using connect_result_t = invoke_result_t<decltype(connect), S, R>;

template<class, class> struct as-receiver; // exposition only

template<class, class> struct as-invocable; // exposition only

// Concepts:

template<class T, class E = exception_ptr>
    concept receiver = see-below;

template<class T, class... An>
    concept receiver_of = see-below;

template<class R, class... An>
    inline constexpr bool is_nothrow_receiver_of_v =
        receiver_of<R, An...> &&
        is_nothrow_invocable_v<decltype(set_value), R, An...>;

template<class O>
    concept operation_state = see-below;

template<class S>
    concept sender = see-below;

template<class S>
    concept typed_sender = see-below;

template<class S, class R>
    concept sender_to = see-below;

template<class S>
    concept scheduler = see-below;

template<class E>
    concept executor = see-below;

template<class E, class F>
    concept executor_of = see-below;

// Sender and receiver utilities type
namespace unspecified { struct sender_base {}; }
using unspecified::sender_base;

template<class S> struct sender_traits;

// Associated execution context property:

struct context_t;

constexpr context_t context;

// Blocking properties:

struct blocking_t;

constexpr blocking_t blocking;

// Properties to allow adaptation of blocking and directionality:

struct blocking_adaptation_t;

constexpr blocking_adaptation_t blocking_adaptation;

// Properties to indicate if submitted tasks represent continuations:

struct relationship_t;

constexpr relationship_t relationship;

// Properties to indicate likely task submission in the future:

struct outstanding_work_t;

constexpr outstanding_work_t outstanding_work;

```

```

// Properties for bulk execution guarantees:

struct bulk_guarantee_t;

constexpr bulk_guarantee_t bulk_guarantee;

// Properties for mapping of execution on to threads:

struct mapping_t;

constexpr mapping_t mapping;

// Memory allocation properties:

template <typename ProtoAllocator>
struct allocator_t;

constexpr allocator_t<void> allocator;

// Executor type traits:

template<class Executor> struct executor_shape;
template<class Executor> struct executor_index;

template<class Executor> using executor_shape_t = typename executor_shape<Executor>::type;
template<class Executor> using executor_index_t = typename executor_index<Executor>::type;

// Polymorphic executor support:

class bad_executor;

template <class... SupportableProperties> class any_executor;

template<class Property> struct prefer_only;

} // namespace execution
} // namespace std

```

2.2 Requirements

2.2.1 ProtoAllocator requirements

A type `A` meets the `ProtoAllocator` requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. [*Note:* For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. –*end note*] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

2.2.2 Invocable archetype

The name `execution::invocable_archetype` is an implementation-defined type such that `invocable<execution::invocable_archetype&>` is true.

A program that creates an instance of `execution::invocable_archetype` is ill-formed.

2.2.3 Customization points

2.2.3.1 `execution::set_value`

The name `execution::set_value` denotes a customization point object. The expression `execution::set_value(R, Vs...)` for some subexpressions `R` and `Vs...` is expression-equivalent to:

- `R.set_value(Vs...)`, if that expression is valid. If the function selected does not send the value(s) `Vs...` to the receiver `R`'s value channel, the program is ill-formed with no diagnostic required.
- Otherwise, `set_value(R, Vs...)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void set_value();
```

and that does not include a declaration of `execution::set_value`. If the function selected by overload resolution does not

send the value(s) $Vs \dots$ to the receiver R 's value channel, the program is ill-formed with no diagnostic required.

- Otherwise, `execution::set_value(R, Vs...)` is ill-formed.

[*Editorial note:* We should probably define what “send the value(s) $Vs \dots$ to the receiver R 's value channel” means more carefully. –*end editorial note*]

2.2.3.2 `execution::set_done`

The name `execution::set_done` denotes a customization point object. The expression `execution::set_done(R)` for some subexpression R is expression-equivalent to:

- `R.set_done()`, if that expression is valid. If the function selected does not signal the receiver R 's done channel, the program is ill-formed with no diagnostic required.
- Otherwise, `set_done(R)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void set_done();
```

and that does not include a declaration of `execution::set_done`. If the function selected by overload resolution does not signal the receiver R 's done channel, the program is ill-formed with no diagnostic required.

- Otherwise, `execution::set_done(R)` is ill-formed.

[*Editorial note:* We should probably define what “signal receiver R 's done channel” means more carefully. –*end editorial note*]

2.2.3.3 `execution::set_error`

The name `execution::set_error` denotes a customization point object. The expression `execution::set_error(R, E)` for some subexpressions R and E are expression-equivalent to:

- `R.set_error(E)`, if that expression is valid. If the function selected does not send the error E to the receiver R 's error channel, the program is ill-formed with no diagnostic required.
- Otherwise, `set_error(R, E)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void set_error();
```

and that does not include a declaration of `execution::set_error`. If the function selected by overload resolution does not send the error E to the receiver R 's error channel, the program is ill-formed with no diagnostic required.

- Otherwise, `execution::set_error(R, E)` is ill-formed.

[*Editorial note:* We should probably define what “send the error E to the receiver R 's error channel” means more carefully. –*end editorial note*]

2.2.3.4 `execution::execute`

The name `execution::execute` denotes a customization point object.

For some subexpressions e and f , let E be a type such that `decltype((e))` is E and let F be a type such that `decltype((f))` is F . The expression `execution::execute(e, f)` is ill-formed if F does not model invocable, or if E does not model either executor or sender. Otherwise, it is expression-equivalent to:

- `e.execute(f)`, if that expression is valid. If the function selected does not execute the function object f on the executor e , the program is ill-formed with no diagnostic required.
- Otherwise, `execute(e, f)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void execute();
```

and that does not include a declaration of `execution::execute`. If the function selected by overload resolution does not execute the function object f on the executor e , the program is ill-formed with no diagnostic required.

- Otherwise, if F is not an instance of `as-invocable<R, E>` for some type R , and `invocable<remove_cvref_t<F>&& sender_to<E, as-receiver<remove_cvref_t<F>, E>>` is true, `execution::submit(e, as-receiver<remove_cvref_t<F>, E)`

{std::forward<F>(f)}), where *as-receiver* is some implementation-defined class template equivalent to:

```
template<class F, class>
struct as-receiver {
    F f_;
    void set_value() noexcept(is_nothrow_invocable_v<F&>) {
        invoke(f_);
    }
    [[noreturn]] void set_error(std::exception_ptr) noexcept {
        terminate();
    }
    void set_done() noexcept {}
};
```

[Editorial note: We should probably define what “execute the function object *F* on the executor *E*” means more carefully. –end editorial note]

2.2.3.5 execution::connect

The name `execution::connect` denotes a customization point object. For some subexpressions *s* and *r*, let *S* be a type such that `decltype((s))` is *S* and let *R* be a type such that `decltype((r))` is *R*. The expression `execution::connect(s, r)` is expression-equivalent to:

- `s.connect(r)`, if that expression is valid, if its type satisfies `operation_state`, and if *S* satisfies `sender`.
- Otherwise, `connect(s, r)`, if that expression is valid, if its type satisfies `operation_state`, and if *S* satisfies `sender`, with overload resolution performed in a context that includes the declaration

```
void connect();
```

and that does not include a declaration of `execution::connect`.

- Otherwise, *as-operation*{*s*, *r*}, if *r* is not an instance of *as-receiver*<*F*, *S*> for some type *F*, and if `receiver_of<R> && executor-of-impl<remove_cvref_t<S>, as-invocable<remove_cvref_t<R>, S>>` is true, where *as-operation* is an implementation-defined class equivalent to

```
struct as-operation {
    remove_cvref_t<S> e_;
    remove_cvref_t<R> r_;
    void start() noexcept try {
        execution::execute(std::move(e_), as-invocable<remove_cvref_t<R>, S>{r_});
    } catch(...) {
        execution::set_error(std::move(r_), current_exception());
    }
};
```

and *as-invocable* is a class template equivalent to the following:

```
template<class R, class>
struct as-invocable {
    R* r_;
    explicit as-invocable(R& r) noexcept
        : r_(std::addressof(r)) {}
    as-invocable(as-invocable&& other) noexcept
        : r_(std::exchange(other.r_, nullptr)) {}
    ~as-invocable() {
        if(r_)
            execution::set_done(std::move(*r_));
    }
    void operator()() & noexcept try {
        execution::set_value(std::move(*r_));
        r_ = nullptr;
    } catch(...) {
        execution::set_error(std::move(*r_), current_exception());
        r_ = nullptr;
    }
};
```

- Otherwise, `execution::connect(s, r)` is ill-formed.

2.2.3.6 execution::start

The name `execution::start` denotes a customization point object. The expression `execution::start(0)` for some lvalue subexpression *o* is expression-equivalent to:

- `o.start()`, if that expression is valid.
- Otherwise, `start(0)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void start();
```

and that does not include a declaration of `execution::start`.

- Otherwise, `execution::start(0)` is ill-formed.

2.2.3.7 `execution::submit`

The name `execution::submit` denotes a customization point object.

For some subexpressions `s` and `r`, let `S` be a type such that `decltype((s))` is `S` and let `R` be a type such that `decltype((r))` is `R`. The expression `execution::submit(s, r)` is ill-formed if `sender_to<S, R>` is not true. Otherwise, it is expression-equivalent to:

- `s.submit(r)`, if that expression is valid and `s` models `sender`. If the function selected does not submit the receiver object `r` via the sender `s`, the program is ill-formed with no diagnostic required.
- Otherwise, `submit(s, r)`, if that expression is valid and `s` models `sender`, with overload resolution performed in a context that includes the declaration

```
void submit();
```

and that does not include a declaration of `execution::submit`. If the function selected by overload resolution does not submit the receiver object `r` via the sender `s`, the program is ill-formed with no diagnostic required.

- Otherwise, `execution::start((newsubmit-receiver<S, R>{s,r})->state_)`, where *submit-receiver* is an implementation-defined class template equivalent to

```
template<class S, class R>
struct submit-receiver {
    struct wrap {
        submit-receiver * p_;
        template<class... As>
            requires receiver_of<R, As...>
        void set_value(As&&... as) && noexcept(is_nothrow_receiver_of_v<R, As...>) {
            execution::set_value(std::move(p_->r_), (As&&) as...);
            delete p_;
        }
        template<class E>
            requires receiver<R, E>
        void set_error(E&& e) && noexcept {
            execution::set_error(std::move(p_->r_), (E&&) e);
            delete p_;
        }
        void set_done() && noexcept {
            execution::set_done(std::move(p_->r_));
            delete p_;
        }
    };
};
remove_cvref_t<R> r_;
connect_result_t<S, wrap> state_;
submit-receiver(S&& s, R&& r)
    : r_((R&&) r)
    , state_(execution::connect((S&&) s, wrap{this})) {}
};
```

2.2.3.8 `execution::schedule`

The name `execution::schedule` denotes a customization point object. For some subexpression `s`, let `s` be a type such that `decltype((s))` is `S`. The expression `execution::schedule(s)` is expression-equivalent to:

- `s.schedule()`, if that expression is valid and its type models `sender`.
- Otherwise, `schedule(s)`, if that expression is valid and its type models `sender` with overload resolution performed in a context that includes the declaration

```
void schedule();
```

and that does not include a declaration of `execution::schedule`.

- Otherwise, `as-sender<remove_cvref_t<S>>>{s}` if `S` satisfies executor, where `as-sender` is an implementation-defined class template equivalent to

```
template<class E>
struct as-sender {
private:
    E ex_;
public:
    template<template<class...> class Tuple, template<class...> class Variant>
        using value_types = Variant<Tuple<>>;
    template<template<class...> class Variant>
        using error_types = Variant<std::exception_ptr>;
    static constexpr bool sends_done = true;

    explicit as-sender(E e) noexcept
        : ex_((E&&) e) {}
    template<class R>
        requires receiver_of<R>
    connect_result_t<E, R> connect(R&& r) && {
        return execution::connect((E&&) ex_, (R&&) r);
    }
    template<class R>
        requires receiver_of<R>
    connect_result_t<const E &, R> connect(R&& r) const & {
        return execution::connect(ex_, (R&&) r);
    }
};
```

- Otherwise, `execution::schedule(s)` is ill-formed.

2.2.3.9 `execution::bulk_execute`

The name `execution::bulk_execute` denotes a customization point object. If `is_convertible_v<N, size_t>` is true, then the expression `execution::bulk_execute(S, F, N)` for some subexpressions `S`, `F`, and `N` is expression-equivalent to:

- `S.bulk_execute(F, N)`, if that expression is valid. If the function selected does not execute `N` invocations of the function object `F` on the executor `s` in bulk with forward progress guarantee `std::query(S, execution::bulk_guarantee)`, and the result of that function does not model `sender<void>`, the program is ill-formed with no diagnostic required.
- Otherwise, `bulk_execute(S, F, N)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
void bulk_execute();
```

and that does not include a declaration of `execution::bulk_execute`. If the function selected by overload resolution does not execute `N` invocations of the function object `F` on the executor `s` in bulk with forward progress guarantee `std::query(E, execution::bulk_guarantee)`, and the result of that function does not model `sender<void>`, the program is ill-formed with no diagnostic required.

- Otherwise, if the types `F` and `executor_index_t<remove_cvref_t<S>>` model invocable and if `std::query(S, execution::bulk_guarantee) equals execution::bulk_guarantee.unsequenced`, then
 - Evaluates `DECAY_COPY(std::forward<decltype(F)>(F))` on the calling thread to create a function object `cf`. [*Note:* Additional copies of `cf` may subsequently be created. *—end note.*]
 - For each value of `i` in `N`, `cf(i)` (or copy of `cf`) will be invoked at most once by an execution agent that is unique for each value of `i`.
 - May block pending completion of one or more invocations of `cf`.
 - Synchronizes with (C++Std [intro.multithread]) the invocations of `cf`.
- Otherwise, `execution::bulk_execute(S, F, N)` is ill-formed.

[*Editorial note:* We should probably define what “execute `N` invocations of the function object `F` on the executor `s` in bulk” means more carefully. *—end editorial note*]

2.2.4 Concepts `receiver` and `receiver_of`

A receiver represents the continuation of an asynchronous operation. An asynchronous operation may complete with a (possibly empty) set of values, an error, or it may be cancelled. A receiver has three principal operations corresponding to the three ways an asynchronous operation may complete: `set_value`, `set_error`, and `set_done`. These are collectively known as a receiver’s *completion-signal operations*.


```

template<class T, class E = exception_ptr>
concept receiver =
    move_constructible<remove_cvref_t<T>> &&
    constructible_from<remove_cvref_t<T>, T> &&
    requires(remove_cvref_t<T>&& t, E&& e) {
        { execution::set_done(std::move(t)) } noexcept;
        { execution::set_error(std::move(t), (E&&) e) } noexcept;
    };

template<class T, class... An>
concept receiver_of =
    receiver<T> &&
    requires(remove_cvref_t<T>&& t, An&&... an) {
        execution::set_value(std::move(t), (An&&) an...);
    };

```

The receiver’s completion-signal operations have semantic requirements that are collectively known as the *receiver contract*, described below:

- None of a receiver’s completion-signal operations shall be invoked before `execution::start` has been called on the operation state object that was returned by `execution::connect` to connect that receiver to a sender.
- Once `execution::start` has been called on the operation state object, exactly one of the receiver’s completion-signal operations shall complete non-exceptionally before the receiver is destroyed.
- If `execution::set_value` exits with an exception, it is still valid to call `execution::set_error` OR `execution::set_done` on the receiver.

Once one of a receiver’s completion-signal operations has completed non-exceptionally, the receiver contract has been satisfied.

2.2.5 Concept `operation_state`

```

template<class O>
concept operation_state =
    destructible<O> &&
    is_object_v<O> &&
    requires (O& o) {
        { execution::start(o) } noexcept;
    };

```

An object whose type satisfies `operation_state` represents the state of an asynchronous operation. It is the result of calling `execution::connect` with a sender and a receiver.

`execution::start` may be called on an `operation_state` object at most once. Once `execution::start` has been invoked, the caller shall ensure that the start of a non-exceptional invocation of one of the receiver’s completion-signalling operations strongly happens before [intro.multithread] the call to the `operation_state` destructor.

The start of the invocation of `execution::start` shall strongly happen before [intro.multithread] the invocation of one of the three receiver operations.

`execution::start` may or may not block pending the successful transfer of execution to one of the three receiver operations.

2.2.6 Concepts `sender` and `sender_to`

XXX TODO The `sender` and `sender_to` concepts...

```

template<class S>
concept sender =
    move_constructible<remove_cvref_t<S>> &&
    !requires {
        typename sender_traits<remove_cvref_t<S>>::__unspecialized; // exposition only
    };

template<class S, class R>
concept sender_to =
    sender<S> &&
    receiver<R> &&
    requires (S&& s, R&& r) {
        execution::connect((S&&) s, (R&&) r);
    };

```

None of these operations shall introduce data races as a result of concurrent invocations of those functions from different threads.

A sender type’s destructor shall not block pending completion of the submitted function objects. [*Note*: The ability to wait for completion of submitted function objects may be provided by the associated execution context. –*end note*]

2.2.7 Concept `typed_sender`

A sender is *typed* if it declares what types it sends through a receiver’s channels. The `typed_sender` concept is defined as:

```
template<template<template<class...> class Tuple, template<class...> class Variant> class>
    struct has-value-types; // exposition only

template<template<class...> class Variant>
    struct has-error-types; // exposition only

template<class S>
    concept has-sender-types = // exposition only
        requires {
            typename has-value-types<S::template value_types>;
            typename has-error-types<S::template error_types>;
            typename bool_constant<S::sends_done>;
        };

template<class S>
    concept typed_sender =
        sender<S> &&
        has-sender-types<sender_traits<remove_cvref_t<S>>>>;
```

2.2.8 Concept `scheduler`

XXX TODO The scheduler concept...

```
template<class S>
    concept scheduler =
        copy_constructible<remove_cvref_t<S>> &&
        equality_comparable<remove_cvref_t<S>> &&
        requires(E&& e) {
            execution::schedule((E&&)e);
        };
```

None of a scheduler’s copy constructor, destructor, equality comparison, or `swap` operation shall exit via an exception.

None of these operations, nor an scheduler type’s `schedule` function, or associated query functions shall introduce data races as a result of concurrent invocations of those functions from different threads.

For any two (possibly `const`) values `x1` and `x2` of some scheduler type `X`, `x1 == x2` shall return `true` only if `x1.query(p) == x2.query(p)` for every property `p` where both `x1.query(p)` and `x2.query(p)` are well-formed and result in a non-void type that is `EqualityComparable` (C++Std [equalitycomparable]). [*Note*: The above requirements imply that `x1 == x2` returns `true` if `x1` and `x2` can be interchanged with identical effects. An scheduler may conceptually contain additional properties which are not exposed by a named property type that can be observed via `execution::query`; in this case, it is up to the concrete scheduler implementation to decide if these properties affect equality. Returning `false` does not necessarily imply that the effects are not identical. –*end note*]

An scheduler type’s destructor shall not block pending completion of any receivers submitted to the sender objects returned from `schedule`. [*Note*: The ability to wait for completion of submitted function objects may be provided by the execution context that produced the scheduler. –*end note*]

In addition to the above requirements, type `s` models `scheduler` only if it satisfies the requirements in the Table below.

In the Table below,

- `s` denotes a (possibly `const`) scheduler object of type `s`,
- `N` denotes a type that models `sender`, and
- `n` denotes a sender object of type `N`

Expression	Return Type	Operational semantics
<code>execution::schedule(s)</code>	<code>N</code>	Evaluates <code>execution::schedule(s)</code> on the calling thread to create <code>N</code> .

`execution::start(o)`, where `o` is the result of a call to `execution::connect(N, r)` for some receiver object `r`, is required to eagerly submit `r` for execution on an execution agent that `s` creates for it. Let `rc` be `r` or an object created by copy or move construction from `r`. The semantic constraints on the sender `N` returned from a scheduler `s`’s `schedule` function are as follows:

- If `rc`'s `set_error` function is called in response to a submission error, scheduling error, or other internal error, let `E` be an expression that refers to that error if `set_error(rc, E)` is well-formed; otherwise, let `E` be an `exception_ptr` that refers to that error. [*Note*: `E` could be the result of calling `current_exception` OR `make_exception_ptr` — *end note*] The scheduler calls `set_error(rc, E)` on an unspecified weakly-parallel execution agent ([*Note*: An invocation of `set_error` on a receiver is required to be `noexcept` — *end note*]), and
- If `rc`'s `set_error` function is called in response to an exception that propagates out of the invocation of `set_value` on `rc`, let `E` be `make_exception_ptr(receiver_invocation_error{})` invoked from within a catch clause that has caught the exception. The executor calls `set_error(rc, E)` on an unspecified weakly-parallel execution agent, and
- A call to `set_done(rc)` is made on an unspecified weakly-parallel execution agent ([*Note*: An invocation of a receiver's `set_done` function is required to be `noexcept` — *end note*]).

[*Note*: The senders returned from a scheduler's `schedule` function have wide discretion when deciding which of the three receiver functions to call upon submission. — *end note*]

2.2.9 Concepts `executor` and `executor_of`

XXX TODO The executor and executor_of concepts...

Let *executor-of-impl* be the exposition-only concept

```
template<class E, class F>
concept executor-of-impl =
    invocable<remove_cvref_t<F>> &&
    constructible_from<remove_cvref_t<F>, F> &&
    move_constructible<remove_cvref_t<F>> &&
    copy_constructible<E> &&
    is_nothrow_copy_constructible_v<E> &&
    equality_comparable<E> &&
    requires(const E& e, F&& f) {
        execution::execute(e, (F&&)f);
    };
```

Then,

```
template<class E>
concept executor =
    executor-of-impl<E, execution::invocable_archetype>;

template<class E, class F>
concept executor_of =
    executor<E> &&
    executor-of-impl<E, F>;
```

Neither of an executor's equality comparison or `swap` operation shall exit via an exception.

None of an executor type's copy constructor, destructor, equality comparison, `swap` function, `execute` function, or associated query functions shall introduce data races as a result of concurrent invocations of those functions from different threads.

For any two (possibly `const`) values `x1` and `x2` of some executor type `X`, `x1 == x2` shall return `true` only if `std::query(x1,p) == std::query(x2,p)` for every property `p` where both `std::query(x1,p)` and `std::query(x2,p)` are well-formed and result in a non-void type that is `equality_comparable` (C++Std [equalitycomparable]). [*Note*: The above requirements imply that `x1 == x2` returns `true` if `x1` and `x2` can be interchanged with identical effects. An executor may conceptually contain additional properties which are not exposed by a named property type that can be observed via `std::query`; in this case, it is up to the concrete executor implementation to decide if these properties affect equality. Returning `false` does not necessarily imply that the effects are not identical. —*end note*]

An executor type's destructor shall not block pending completion of the submitted function objects. [*Note*: The ability to wait for completion of submitted function objects may be provided by the associated execution context. —*end note*]

In addition to the above requirements, types `E` and `F` model `executor_of` only if they satisfy the requirements of the Table below.

In the Table below,

- `e` denotes a (possibly `const`) executor object of type `E`,
- `cf` denotes the function object `DECAY_COPY(std::forward<F>(f))`
- `f` denotes a function of type `F&&` invocable as `cf()` and where `decay_t<F>` models `move_constructible`.

Expression	Return Type	Operational semantics
<code>e == x</code>	<code>bool</code>	Evaluates <code>DECAY_COPY(std::forward<E>(f))</code> on the calling

```
execution::execute(e, f)          void
```

Evaluates `bool_constant{std::forward<f>() }` on the calling thread to create `cf` that will be invoked at most once by an execution agent.
 May block pending completion of this invocation.
 Synchronizes with `[intro.multithread]` the invocation of `f`.
 Shall not propagate any exception thrown by the function object or any other function submitted to the executor.
 [Note: The treatment of exceptions thrown by one-way submitted functions is implementation-defined. The forward progress guarantee of the associated execution agent(s) is implementation-defined. —end note.]

[Editorial note: The operational semantics of `execution::execute` should be specified with the `execution::execute` CPO rather than the `executor` concept. —end note.]

2.2.10 Sender and receiver traits

2.2.10.1 Class template `sender_traits`

XXX TODO The class template `sender_traits`...

The class template `sender_traits` can be used to query information about a sender; in particular, what values and errors it sends through a receiver's value and error channel, and whether or not it ever calls `set_done` on a receiver.

The primary `sender_traits<S>` class template is defined as if inheriting from an implementation-defined class template `sender_traits-base<S>` defined as follows:

- Let *has-sender-types* be an implementation-defined concept equivalent to:

```
template<template<template<class...> class, template<class...> class> class>
    struct has-value-types ; // exposition only

template<template<template<class...> class> class>
    struct has-error-types ; // exposition only

template<class S>
    concept has-sender-types =
        requires {
            typename has-value-types <S::template value_types>;
            typename has-error-types <S::template error_types>;
            typename bool_constant<S::sends_done>;
        };
```

If *has-sender-types<S>* is true, then *sender_traits-base* is equivalent to:

```
template<class S>
    struct sender_traits-base {
        template<template<class...> class Tuple, template<class...> class Variant>
            using value_types = typename S::template value_types<Tuple, Variant>;

        template<template<class...> class Variant>
            using error_types = typename S::template error_types<Variant>;

        static constexpr bool sends_done = S::sends_done;
    };
```

- Otherwise, let *void-receiver* be an implementation-defined class type equivalent to

```
struct void-receiver { // exposition only
    void set_value() noexcept;
    void set_error(exception_ptr) noexcept;
    void set_done() noexcept;
};
```

If *executor-of-impl<S, as-invocable<void-receiver, S>>* is true, then *sender_traits-base* is equivalent to

```
template<class S>
    struct sender_traits-base {
        template<template<class...> class Tuple, template<class...> class Variant>
            using value_types = Variant<Tuple<>>>;

        template<template<class...> class Variant>
            using error_types = Variant<exception_ptr>;
```

```
static constexpr bool sends_done = true;
};
```

- Otherwise, if `derived_from<S, sender_base>` is true, then *sender-traits-base* is equivalent to

```
template<class S>
struct sender_traits_base {};
```

- Otherwise, *sender-traits-base* is equivalent to

```
template<class S>
struct sender_traits_base {
    using __unspecialized = void; // exposition only
};
```

Because a sender may send one set of types or another to a receiver based on some runtime condition, `sender_traits` may provide a nested `value_types` template that is parameterized on a tuple-like class template and a variant-like class template that are used to hold the result.

[*Example:* If a sender type `S` sends types `As...` or `Bs...` to a receiver's value channel, it may specialize `sender_traits` such that `typename sender_traits<S>::value_types<tuple, variant>` names the type `variant<tuple<As...>, tuple<Bs...>>` – *end example*]

Because a sender may send one or another type of error types to a receiver, `sender_traits` may provide a nested `error_types` template that is parameterized on a variant-like class template that is used to hold the result.

[*Example:* If a sender type `S` sends error types `exception_ptr` or `error_code` to a receiver's error channel, it may specialize `sender_traits` such that `typename sender_traits<S>::error_types<variant>` names the type `variant<exception_ptr, error_code>` – *end example*]

A sender type can signal that it never calls `set_done` on a receiver by specializing `sender_traits` such that `sender_traits<S>::sends_done` is false; conversely, it may set `sender_traits<S>::sends_done` to true to indicate that it does call `set_done` on a receiver.

Users may specialize `sender_traits` on program-defined types.

2.2.11 Query-only properties

2.2.11.1 Associated execution context property

```
struct context_t
{
    template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    using polymorphic_query_result_type = any;

    template<class Executor>
        static constexpr decltype(auto) static_query_v
            = Executor::query(context_t());
};
```

The `context_t` property can be used only with `query`, which returns the execution context associated with the executor.

The value returned from `std::query(e, context_t)`, where `e` is an executor, shall not change between invocations.

2.2.11.2 Polymorphic executor wrappers

The `any_executor` class template provides polymorphic wrappers for executors.

In several places in this section the operation `CONTAINS_PROPERTY(p, pn)` is used. All such uses mean `std::disjunction_v<std::is_same<p, pn>...>`.

In several places in this section the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is used. All such uses mean the first type `P` in the parameter pack `pn` for which `std::is_convertible_v<p, P>` is true. If no such type `P` exists, the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is ill-formed.

```
template <class... SupportableProperties>
class any_executor
```

```

{
public:
    // construct / copy / destroy:

    any_executor() noexcept;
    any_executor(nullptr_t) noexcept;
    any_executor(const any_executor& e) noexcept;
    any_executor(any_executor&& e) noexcept;
    template<class... OtherSupportableProperties>
        any_executor(any_executor<OtherSupportableProperties...> e);
    template<class... OtherSupportableProperties>
        any_executor(any_executor<OtherSupportableProperties...> e) = delete;
    template<executor Executor>
        any_executor(Executor e);

    any_executor& operator=(const any_executor& e) noexcept;
    any_executor& operator=(any_executor&& e) noexcept;
    any_executor& operator=(nullptr_t) noexcept;
    template<executor Executor>
        any_executor& operator=(Executor e);

    ~any_executor();

    // any_executor modifiers:

    void swap(any_executor& other) noexcept;

    // any_executor operations:

    template <class Property>
    any_executor require(Property) const;

    template <class Property>
    typename Property::polymorphic_query_result_type query(Property) const;

    template<class Function>
        void execute(Function&& f) const;

    // any_executor capacity:

    explicit operator bool() const noexcept;

    // any_executor target access:

    const type_info& target_type() const noexcept;
    template<executor Executor> Executor* target() noexcept;
    template<executor Executor> const Executor* target() const noexcept;
};

// any_executor comparisons:

template <class... SupportableProperties>
bool operator==(const any_executor<SupportableProperties...> a, const any_executor<SupportableProperties...> b) noexcept;
template <class... SupportableProperties>
bool operator==(const any_executor<SupportableProperties...> e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator==(nullptr_t, const any_executor<SupportableProperties...> e) noexcept;
template <class... SupportableProperties>
bool operator!=(const any_executor<SupportableProperties...> a, const any_executor<SupportableProperties...> b) noexcept;
template <class... SupportableProperties>
bool operator!=(const any_executor<SupportableProperties...> e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator!=(nullptr_t, const any_executor<SupportableProperties...> e) noexcept;

// any_executor specialized algorithms:

template <class... SupportableProperties>
void swap(any_executor<SupportableProperties...> a, any_executor<SupportableProperties...> b) noexcept;

template <class Property, class... SupportableProperties>
any_executor prefer(const any_executor<SupportableProperties>& e, Property p);

```

The `any_executor` class satisfies the executor concept requirements.

[*Note:* To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more `any_executor` objects. —*end note*]

Each property type in the `SupportableProperties...` pack shall provide a nested type `polymorphic_query_result_type`.

The *target* is the executor object that is held by the wrapper.

2.2.11.2.1 any_executor constructors

```
any_executor() noexcept;
```

Postconditions: !*this.

```
any_executor(nullptr_t) noexcept;
```

Postconditions: !*this.

```
any_executor(const any_executor& e) noexcept;
```

Postconditions: !*this if !e; otherwise, *this targets e.target() or a copy of e.target().

```
any_executor(any_executor&& e) noexcept;
```

Effects: If !e, *this has no target; otherwise, moves e.target() or move-constructs the target of e into the target of *this, leaving e in a valid state with an unspecified value.

```
template<class... OtherSupportableProperties>
    any_executor(any_executor<OtherSupportableProperties...> e);
```

Remarks: This function shall not participate in overload resolution unless: * CONTAINS_PROPERTY(p, OtherSupportableProperties), where p is each property in SupportableProperties....

Effects: *this targets a copy of e initialized with std::move(e).

```
template<class... OtherSupportableProperties>
    any_executor(any_executor<OtherSupportableProperties...> e) = delete;
```

Remarks: This function shall not participate in overload resolution unless CONTAINS_PROPERTY(p, OtherSupportableProperties) is false for some property p in SupportableProperties....

```
template<executor Executor>
    any_executor(Executor e);
```

Remarks: This function shall not participate in overload resolution unless:

- can_require_v<Executor, P>, if P::is_requirable, where P is each property in SupportableProperties....
- can_prefer_v<Executor, P>, if P::is_preferable, where P is each property in SupportableProperties....
- and can_query_v<Executor, P>, if P::is_requirable == false and P::is_preferable == false, where P is each property in SupportableProperties....

Effects: *this targets a copy of e.

2.2.11.2.2 any_executor assignment

```
any_executor& operator=(const any_executor& e) noexcept;
```

Effects: any_executor(e).swap(*this).

Returns: *this.

```
any_executor& operator=(any_executor&& e) noexcept;
```

Effects: Replaces the target of *this with the target of e, leaving e in a valid state with an unspecified value.

Returns: *this.

```
any_executor& operator=(nullptr_t) noexcept;
```

Effects: any_executor(nullptr).swap(*this).

Returns: *this.

```
template<executor Executor>
    any_executor& operator=(Executor e);
```

Requires: As for template<executor Executor> any_executor(Executor e).

Effects: any_executor(std::move(e)).swap(*this).

Returns: *this.

2.2.11.2.3 any_executor destructor

```
~any_executor();
```

Effects: If *this != nullptr, releases shared ownership of, or destroys, the target of *this.

2.2.11.2.4 any_executor modifiers

```
void swap(any_executor& other) noexcept;
```

Effects: Interchanges the targets of *this and other.

2.2.11.2.5 any_executor operations

```
template <class Property>
any_executor require(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_requirable` is well-formed and has the value `true`.

Returns: A polymorphic wrapper whose target is the result of `std::require(e, p)`, where `e` is the target object of *this.

```
template <class Property>
typename Property::polymorphic_query_result_type query(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)` is well-formed.

Returns: If `std::query(e, p)` is well-formed, `static_cast<Property::polymorphic_query_result_type>(std::query(e, p))`, where `e` is the target object of *this. Otherwise, `Property::polymorphic_query_result_type{}`.

```
template<class Function>
void execute(Function&& f) const;
```

Effects: Performs `execution::execute(e, f2)`, where:

- `e` is the target object of *this;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when invoked as `f2()`, performs `f1()`.

2.2.11.2.6 any_executor capacity

```
explicit operator bool() const noexcept;
```

Returns: `true` if *this has a target, otherwise `false`.

2.2.11.2.7 any_executor target access

```
const type_info& target_type() const noexcept;
```

Returns: If *this has a target of type `T`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<executor Executor> Executor* target() noexcept;
template<executor Executor> const Executor* target() const noexcept;
```

Returns: If `target_type() == typeid(Executor)` a pointer to the stored executor target; otherwise a null pointer value.

2.2.11.2.8 any_executor comparisons

```
template<class... SupportableProperties>
bool operator==(const any_executor<SupportableProperties...> a, const any_executor<SupportableProperties...> b) noexcept;
```

Returns:

- `true` if `!a` and `!b`;
- `true` if `a` and `b` share a target;
- `true` if `e` and `f` are the same type and `e == f`, where `e` is the target of `a` and `f` is the target of `b`;
- otherwise `false`.

```
template<class... SupportableProperties>
```



```
bool operator==(const any_executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator==(nullptr_t, const any_executor<SupportableProperties...>& e) noexcept;
```

Returns: !e.

```
template<class... SupportableProperties>
bool operator!=(const any_executor<SupportableProperties...>& a, const any_executor<SupportableProperties...>& b) noexcept;
```

Returns: !(a == b).

```
template<class... SupportableProperties>
bool operator!=(const any_executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator!=(nullptr_t, const any_executor<SupportableProperties...>& e) noexcept;
```

Returns: (bool) e.

2.2.11.2.9 any_executor specialized algorithms

```
template<class... SupportableProperties>
void swap(any_executor<SupportableProperties...>& a, any_executor<SupportableProperties...>& b) noexcept;
```

Effects: a.swap(b).

```
template <class Property, class... SupportableProperties>
any_executor prefer(const any_executor<SupportableProperties...>& e, Property p);
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_preferable` is well-formed and has the value true.

Returns: A polymorphic wrapper whose target is the result of `std::prefer(e, p)`, where e is the target object of *this.

2.2.12 Behavioral properties

Behavioral properties define a set of mutually-exclusive nested properties describing executor behavior.

Unless otherwise specified, behavioral property types *s*, their nested property types *s::Ni*, and nested property objects *s::ni* conform to the following specification:

```
struct S
{
    template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;
    using polymorphic_query_result_type = S;

    template<class Executor>
        static constexpr auto static_query_v
            = see-below;

    template<class Executor>
        friend constexpr S query(const Executor& ex, const Property& p) noexcept(see-below);

    friend constexpr bool operator==(const S& a, const S& b);
    friend constexpr bool operator!=(const S& a, const S& b) { return !operator==(a, b); }

    constexpr S();

    struct N1
    {
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;
        using polymorphic_query_result_type = S;

        template<class Executor>
            static constexpr auto static_query_v
                = see-below;

        static constexpr S value() { return S(N1()); }
    };

    static constexpr N1 n1;
```

```
constexpr S(const N1);

...

struct NN
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;
    using polymorphic_query_result_type = S;

    template<class Executor>
        static constexpr auto static_query_v
            = see-below;

    static constexpr S value() { return S(NN()); }
};

static constexpr NN nN;

constexpr S(const NN);
};
```

Queries for the value of an executor’s behavioral property shall not change between invocations unless the executor is assigned another executor with a different value of that behavioral property.

$S()$ and $S(S::Ei())$ are all distinct values of S . [*Note: This means they compare unequal. –end note.*]

The value returned from $\text{std::query}(e1, p1)$ and a subsequent invocation $\text{std::query}(e1, p1)$, where

- $p1$ is an instance of S or $S::Ei$, and
- $e2$ is the result of $\text{std::require}(e1, p2)$ OR $\text{std::prefer}(e1, p2)$,

shall compare equal unless

- $p2$ is an instance of $S::Ei$, and
- $p1$ and $p2$ are different types.

The value of the expression $S::N1::\text{static_query_v}<\text{Executor}>$ is

- $\text{Executor}::\text{query}(S::N1())$, if that expression is a well-formed expression;
- ill-formed if $\text{declval}<\text{Executor}>().\text{query}(S::N1())$ is well-formed;
- ill-formed if $\text{can_query_v}<\text{Executor}, S::Ni>$ is true for any $1 < i \leq N$;
- otherwise $S::N1()$.

[*Note: These rules automatically enable the $S::N1$ property by default for executors which do not provide a query function for properties $S::Ni$. –end note*]

The value of the expression $S::Ni::\text{static_query_v}<\text{Executor}>$, for all $1 < i \leq N$, is

- $\text{Executor}::\text{query}(S::Ni())$, if that expression is a well-formed constant expression;
- otherwise ill-formed.

The value of the expression $S::\text{static_query_v}<\text{Executor}>$ is

- $\text{Executor}::\text{query}(S())$, if that expression is a well-formed constant expression;
- otherwise, ill-formed if $\text{declval}<\text{Executor}>().\text{query}(S())$ is well-formed;
- otherwise, $S::Ni::\text{static_query_v}<\text{Executor}>$ for the least $i \leq N$ for which this expression is a well-formed constant expression;
- otherwise ill-formed.

[*Note: These rules automatically enable the $S::N1$ property by default for executors which do not provide a query function for properties S or $S::Ni$. –end note*]

Let k be the least value of i for which $\text{can_query_v}<\text{Executor}, S::Ni>$ is true, if such a value of i exists.

```
template<class Executor>
    friend constexpr S query(const Executor& ex, const Property& p) noexcept(noexcept(std::query(ex, std::declval<const S::Nk>())));
```

Returns: $\text{std::query}(ex, S::Nk())$.

Remarks: This function shall not participate in overload resolution unless $\text{is_same_v}<\text{Property}, S> \ \&\& \ \text{can_query_v}<\text{Executor}, S::Ni>$

is true for at least one $S : Ni$.

```
bool operator==(const S& a, const S& b);
```

Returns: true if a and b were constructed from the same constructor; false, otherwise.

2.2.12.1 Blocking properties

The `blocking_t` property describes what guarantees executors provide about the blocking behavior of their execution functions.

`blocking_t` provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
<code>blocking_t::possibly_t</code>	<code>blocking.possibly</code>	Invocation of an executor’s execution function may block pending completion of one or more invocations of the submitted function object.
<code>blocking_t::always_t</code>	<code>blocking.always</code>	Invocation of an executor’s execution function shall block until completion of all invocations of submitted function object.
<code>blocking_t::never_t</code>	<code>blocking.never</code>	Invocation of an executor’s execution function shall not block pending completion of the invocations of the submitted function object.

2.2.12.1.1 `blocking_t::always_t` customization points

In addition to conforming to the above specification, the `blocking_t::always_t` property provides the following customization:

```
struct always_t
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = false;

    template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

    template<class Executor>
        friend see-below require(Executor ex, blocking_t::always_t);
};
```

If the executor has the `blocking_adaptation_t::allowed_t` property, this customization uses an adapter to implement the `blocking_t::always_t` property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_t::always_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` provides an overload of `require` such that `e1.require(blocking.always)` returns a copy of `e1`, an overload of `query` such that `std::query(e1,blocking)` returns `blocking.always`, and functions `execute` and `bulk_execute` shall block the calling thread until the submitted functions have finished execution. `e1` has the same executor properties as `ex`, except for the addition of the `blocking_t::always_t` property, and removal of `blocking_t::never_t` and `blocking_t::possibly_t` properties if present.

Remarks: This function shall not participate in overload resolution unless `blocking_adaptation_t::static_query_v<Executor>` is `blocking_adaptation.allowed`.

2.2.12.2 Properties to indicate if blocking and directionality may be adapted

The `blocking_adaptation_t` property allows or disallows blocking or directionality adaptation via `std::require`.

blocking_adaptation_t provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
blocking_adaptation_t::disallowed_t	blocking_adaptation.disallowed	The require customization point may not adapt the executor to add the blocking_t::always_t property. The require customization point may adapt the executor to add the blocking_t::always_t property.
blocking_adaptation_t::allowed_t	blocking_adaptation.allowed	

2.2.12.2.1 blocking_adaptation_t::allowed_t customization points

In addition to conforming to the above specification, the blocking_adaptation_t::allowed_t property provides the following customization:

```
struct allowed_t
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = false;

    template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

    template<class Executor>
        friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
};
```

This customization uses an adapter to implement the blocking_adaptation_t::allowed_t property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. In addition, blocking_adaptation_t::static_query_v<E1> is blocking_adaptation.allowed, and e1.require(blocking_adaptation.disallowed) yields a copy of ex. e1 has the same executor properties as ex, except for the addition of the blocking_adaptation_t::allowed_t property.

2.2.12.3 Properties to indicate if submitted tasks represent continuations

The relationship_t property allows users of executors to indicate that submitted tasks represent continuations.

relationship_t provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
relationship_t::fork_t	relationship.fork	Function objects submitted through the executor do not represent continuations of the caller. Function objects submitted through the executor represent continuations of the caller. Invocation of the submitted function object may be deferred until the caller completes.
relationship_t::continuation_t	relationship.continuation	

2.2.12.4 Properties to indicate likely task submission in the future

The `outstanding_work_t` property allows users of executors to indicate that task submission is likely in the future.

`outstanding_work_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
<code>outstanding_work_t::untracked_t</code>	<code>outstanding_work.untracked</code>	The existence of the executor object does not indicate any likely future submission of a function object. The existence of the executor object represents an indication of likely future submission of a function object. The executor or its associated execution context may choose to maintain execution resources in anticipation of this submission.
<code>outstanding_work_t::tracked_t</code>	<code>outstanding_work.tracked</code>	

[*Note:* The `outstanding_work_t::tracked_t` and `outstanding_work_t::untracked_t` properties are used to communicate to the associated execution context intended future work submission on the executor. The intended effect of the properties is the behavior of execution context’s facilities for awaiting outstanding work; specifically whether it considers the existance of the executor object with the `outstanding_work_t::tracked_t` property enabled outstanding work when deciding what to wait on. However this will be largely defined by the execution context implementation. It is intended that the execution context will define its wait facilities and on-destruction behaviour and provide an interface for querying this. An initial work towards this is included in P0737r0. –*end note*]

2.2.12.5 Properties for bulk execution guarantees

Bulk execution guarantee properties communicate the forward progress and ordering guarantees of execution agents associated with the bulk execution.

`bulk_guarantee_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
<code>bulk_guarantee_t::unsequenced_t</code>	<code>bulk_guarantee.unsequenced</code>	Execution agents within the same bulk execution may be parallelized and vectorized.
<code>bulk_guarantee_t::sequenced_t</code>	<code>bulk_guarantee.sequenced</code>	Execution agents within the same bulk execution may not be parallelized.
<code>bulk_guarantee_t::parallel_t</code>	<code>bulk_guarantee.parallel</code>	Execution agents within the same bulk execution may be parallelized.

Execution agents associated with the `bulk_guarantee_t::unsequenced_t` property may invoke the function object in an unordered fashion. Any such invocations in the same thread of execution are unsequenced with respect to each other. [*Note:* This means that multiple execution agents may be interleaved on a single thread of execution, which overrides the usual guarantee from [intro.execution] that function executions do not interleave with one another. –*end note*]

Execution agents associated with the `bulk_guarantee_t::sequenced_t` property invoke the function object in sequence in lexicographic order of their indices.

Execution agents associated with the `bulk_guarantee_t::parallel_t` property invoke the function object with a parallel forward

progress guarantee. Any such invocations in the same thread of execution are indeterminately sequenced with respect to each other. [Note: It is the caller’s responsibility to ensure that the invocation does not introduce data races or deadlocks. –end note]

[Editorial note: The descriptions of these properties were ported from [algorithms.parallel.user]. The intention is that a future standard will specify execution policy behavior in terms of the fundamental properties of their associated executors. We did not include the accompanying code examples from [algorithms.parallel.user] because the examples seem easier to understand when illustrated by `std::for_each`. –end editorial note]

2.2.12.6 Properties for mapping of execution on to threads

The `mapping_t` property describes what guarantees executors provide about the mapping of execution agents onto threads of execution.

`mapping_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
<code>mapping_t::thread_t</code>	<code>mapping.thread</code>	Execution agents are mapped onto threads of execution.
<code>mapping_t::new_thread_t</code>	<code>mapping.new_thread</code>	Each execution agent is mapped onto a new thread of execution.
<code>mapping_t::other_t</code>	<code>mapping.other</code>	Mapping of each execution agent is implementation-defined.

[Note: A mapping of an execution agent onto a thread of execution implies the execution agent runs as-if on a `std::thread`. Therefore, the facilities provided by `std::thread`, such as thread-local storage, are available. `mapping_t::new_thread_t` provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. –end note]

2.2.13 Properties for customizing memory allocation

```
template <typename ProtoAllocator>
struct allocator_t;
```

The `allocator_t` property conforms to the following specification:

```
template <typename ProtoAllocator>
struct allocator_t
{
    template <class T>
        static constexpr bool is_applicable_property_v = executor<T>;

    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;

    template<class Executor>
    static constexpr auto static_query_v
        = Executor::query(allocator_t);

    template <typename OtherProtoAllocator>
    allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const;

    static constexpr ProtoAllocator value() const;

private:
    ProtoAllocator a_; // exposition only
};
```

Property	Notes	Requirements
<code>allocator_t<ProtoAllocator></code>	Result of <code>allocator_t<void>::operator(OtherProtoAllocator)</code> .	The executor shall use the encapsulated allocator to allocate any memory required to store the submitted function object.
<code>allocator_t<void></code>	Specialisation of <code>allocator_t<ProtoAllocator></code> .	The executor shall use an implementation defined default allocator to allocate any memory required to store

the submitted function object.

If the expression `std::query(E, P)` is well formed, where `P` is an object of type `allocator_t<ProtoAllocator>`, then: * the type of the expression `std::query(E, P)` shall satisfy the `ProtoAllocator` requirements; * the result of the expression `std::query(E, P)` shall be the allocator currently established in the executor `E`; and * the expression `std::query(E, allocator_t<void>{})` shall also be well formed and have the same result as `std::query(E, P)`.

2.2.13.1 `allocator_t` members

```
template <typename OtherProtoAllocator>
allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const;
```

Returns: An allocator object whose exposition-only member `a_` is initialized as `a_(a)`.

Remarks: This function shall not participate in overload resolution unless `ProtoAllocator` is void.

[*Note:* It is permitted for `a` to be an executor's implementation-defined default allocator and, if so, the default allocator may also be established within an executor by passing the result of this function to `require`. *—end note*]

```
static constexpr ProtoAllocator value() const;
```

Returns: The exposition-only member `a_`.

Remarks: This function shall not participate in overload resolution unless `ProtoAllocator` is not void.

2.3 Executor type traits

2.3.1 Associated shape type

```
template<class Executor>
struct executor_shape
{
private:
    // exposition only
    template<class T>
    using helper = typename T::shape_type;

public:
    using type = std::experimental::detected_or_t<
        size_t, helper, decltype(std::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

2.3.2 Associated index type

```
template<class Executor>
struct executor_index
{
private:
    // exposition only
    template<class T>
    using helper = typename T::index_type;

public:
    using type = std::experimental::detected_or_t<
        executor_shape_t<Executor>, helper, decltype(std::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

2.4 Polymorphic executor support

2.4.1 Class `bad_executor`

An exception of type `bad_executor` is thrown by polymorphic executor member functions `execute` and `bulk_execute` when the executor object has no target.

```

class bad_executor : public exception
{
public:
    // constructor:
    bad_executor() noexcept;
};

```

2.4.1.1 bad_executor constructors

```
bad_executor() noexcept;
```

Effects: Constructs a bad_executor object.

Postconditions: what() returns an implementation-defined NTBS.

2.4.2 Struct prefer_only

The prefer_only struct is a property adapter that disables the is_requirable value.

[Example:

Consider a generic function that performs some task immediately if it can, and otherwise asynchronously in the background.

```

template<class Executor, class Callback>
void do_async_work(
    Executor ex,
    Callback callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        std::require(ex,
            execution::single,
            execution::oneway,
            ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            std::prefer(ex,
                execution::outstanding_work.tracked),
            callback);
    }
}

```

This function can be used with an inline executor which is defined as follows:

```

struct inline_executor
{
    constexpr bool operator==(const inline_executor&) const noexcept
    {
        return true;
    }

    constexpr bool operator!=(const inline_executor&) const noexcept
    {
        return false;
    }

    template<class Function> void execute(Function f) const noexcept
    {
        f();
    }
};

```

as, in the case of an unsupported property, invocation of std::prefer will fall back to an identity operation.

The polymorphic executor wrapper should be able to simply swap in, so that we could change do_async_work to the non-template function:

```

void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        execution::outstanding_work_t::tracked_t> ex,

```



```

        std::function<void()> callback)
{
    if (try_work() == done)
    {
        // Work completed immediately, invoke callback.
        std::require(ex,
            execution::single,
            execution::oneway,
        ).execute(callback);
    }
    else
    {
        // Perform work in background. Track outstanding work.
        start_background_work(
            std::prefer(ex,
                execution::outstanding_work.tracked),
            callback);
    }
}
}

```

with no change in behavior or semantics.

However, if we simply specify `execution::outstanding_work.tracked` in the executor template parameter list, we will get a compile error due to the executor template not knowing that `execution::outstanding_work.tracked` is intended for use with `prefer` only. At the point of construction from an `inline_executor` called `ex`, `executor` will try to instantiate implementation templates that perform the ill-formed `std::require(ex, execution::outstanding_work.tracked)`.

The `prefer_only` adapter addresses this by turning off the `is_requirable` attribute for a specific property. It would be used in the above example as follows:

```

void do_async_work(
    executor<
        execution::single,
        execution::oneway,
        prefer_only<execution::outstanding_work_t::tracked_t>> ex,
    std::function<void()> callback)
{
    ...
}

```

– *end example*

```

template<class InnerProperty>
struct prefer_only
{
    InnerProperty property;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = InnerProperty::is_preferable;

    using polymorphic_query_result_type = see-below; // not always defined

    template<class Executor>
        static constexpr auto static_query_v = see-below; // not always defined

    constexpr prefer_only(const InnerProperty& p);

    constexpr auto value() const
        noexcept(noexcept(std::declval<const InnerProperty>().value()))
        -> decltype(std::declval<const InnerProperty>().value());

    template<class Executor, class Property>
    friend auto prefer(Executor ex, const Property& p)
        noexcept(noexcept(std::prefer(std::move(ex), std::declval<const InnerProperty>())))
        -> decltype(std::prefer(std::move(ex), std::declval<const InnerProperty>()));

    template<class Executor, class Property>
    friend constexpr auto query(const Executor& ex, const Property& p)
        noexcept(noexcept(std::query(ex, std::declval<const InnerProperty>())))
        -> decltype(std::query(ex, std::declval<const InnerProperty>()));
};

```

If `InnerProperty::polymorphic_query_result_type` is valid and denotes a type, the template instantiation `prefer_only<InnerProperty>` defines a nested type `polymorphic_query_result_type` as a synonym for `InnerProperty::polymorphic_query_result_type`.

If `InnerProperty::static_query_v` is a variable template and `InnerProperty::static_query_v<E>` is well formed for some executor

type `E`, the template instantiation `prefer_only<InnerProperty>` defines a nested variable template `static_query_v` as a synonym for `InnerProperty::static_query_v`.

```
constexpr prefer_only(const InnerProperty& p);
```

Effects: Initializes property with `p`.

```
constexpr auto value() const
    noexcept(noexcept(std::declval<const InnerProperty>().value()))
    -> decltype(std::declval<const InnerProperty>().value());
```

Returns: `property.value()`.

Remarks: Shall not participate in overload resolution unless the expression `property.value()` is well-formed.

```
template<class Executor, class Property>
friend auto prefer(Executor ex, const Property& p)
    noexcept(noexcept(std::prefer(std::move(ex), std::declval<const InnerProperty>())))
    -> decltype(std::prefer(std::move(ex), std::declval<const InnerProperty>()));
```

Returns: `std::prefer(std::move(ex), p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is true, and the expression `std::prefer(std::move(ex), p.property)` is well-formed.

```
template<class Executor, class Property>
friend constexpr auto query(const Executor& ex, const Property& p)
    noexcept(noexcept(std::query(ex, std::declval<const InnerProperty>())))
    -> decltype(std::query(ex, std::declval<const InnerProperty>()));
```

Returns: `std::query(ex, p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is true, and the expression `std::query(ex, p.property)` is well-formed.

2.5 Thread pools

Thread pools manage execution agents which run on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

2.5.1 Header `<thread_pool>` synopsis

```
namespace std {

    class static_thread_pool;

} // namespace std
```

2.5.2 Class `static_thread_pool`

`static_thread_pool` is a statically-sized thread pool which may be explicitly grown via thread attachment. The `static_thread_pool` is expected to be created with the use case clearly in mind with the number of threads known by the creator. As a result, no default constructor is considered correct for arbitrary use cases and `static_thread_pool` does not support any form of automatic resizing.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`'s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` represents work as parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. –*end note.*]

```
class static_thread_pool
{
public:
    using scheduler_type = see-below;
    using executor_type = see-below;

    // construction/destruction
    explicit static_thread_pool(std::size_t num_threads);

    // nocopy
    static_thread_pool(const static_thread_pool&) = delete;
```

```

static_thread_pool& operator=(const static_thread_pool&) = delete;

// stop accepting incoming work and wait for work to drain
~static_thread_pool();

// attach current thread to the thread pools list of worker threads
void attach();

// signal all work to complete
void stop();

// wait for all threads in the thread pool to complete
void wait();

// placeholder for a general approach to getting schedulers from
// standard contexts.
scheduler_type scheduler() noexcept;

// placeholder for a general approach to getting executors from
// standard contexts.
executor_type executor() noexcept;
};

```

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the number of existing executor objects associated with the `static_thread_pool` for which the `execution::outstanding_work.tracked` property is established;
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, scheduler and sender, but not yet invoked; and
- the number of function objects that are currently being invoked within the `static_thread_pool`.

The `static_thread_pool` member functions `scheduler`, `executor`, `attach`, `wait`, and `stop`, and the associated schedulers', senders' and executors' copy constructors and member functions, do not introduce data races as a result of concurrent invocations of those functions from different threads of execution.

A `static_thread_pool`'s threads run execution agents with forward progress guarantee delegation. [*Note:* Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to running execution agents; *i.e.*, execution agents which have run the first step of the function object. *–end note*]

2.5.2.1 Types

```
using scheduler_type = see-below;
```

A scheduler type conforming to the specification for `static_thread_pool` scheduler types described below.

```
using executor_type = see-below;
```

An executor type conforming to the specification for `static_thread_pool` executor types described below.

2.5.2.2 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

2.5.2.3 Worker management

```
void attach();
```

Effects: Adds the calling thread to the pool such that this thread is used to execute submitted function objects. [*Note:* Threads created during thread pool construction, or previously attached to the pool, will continue to be used for function object execution. *–end note*] Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. Invocation of `stop()` returns without waiting for the threads to complete. Subsequent invocations to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is $\emptyset$. Blocks the calling thread (C++Std [defs.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent invocations of `attach()` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

2.5.2.4 Scheduler creation

```
scheduler_type scheduler() noexcept;
```

Returns: A scheduler that may be used to create sender objects that may be used to submit receiver objects to the thread pool. The returned scheduler has the following properties already established:

- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

2.5.2.5 Executor creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::outstanding_work.untracked`
- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

2.5.3 static_thread_pool scheduler types

All scheduler types accessible through `static_thread_pool::scheduler()`, and subsequent invocations of the member function `require`, conform to the following specification.

```
class C
{
public:

    // types:

    using sender_type = see-below;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // scheduler operations:

    see-below require(const execution::allocator_t<void>& a) const;
    template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

    see-below query(execution::context_t) const noexcept;
    see-below query(execution::allocator_t<void>) const noexcept;
    template<class ProtoAllocator>
    see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

    bool running_in_this_thread() const noexcept;
};
```

```
bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
```

Objects of type `c` are associated with a `static_thread_pool`.

2.5.3.1 Constructors

```
C(const C& other) noexcept;
```

Postconditions: `*this == other`.

```
C(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

2.5.3.2 Assignment

```
C& operator=(const C& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
C& operator=(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

2.5.3.3 Operations

```
see-below require(const execution::allocator_t<void>& a) const;
```

Returns: `require(execution::allocator(x))`, where `x` is an implementation-defined default allocator.

```
template<class ProtoAllocator>
see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An scheduler object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned scheduler object are identical to those of `*this`.

```
static_thread_pool& query(execution::context_t) const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;
```

Returns: The allocator object associated with the executor, with type and value as either previously established by the `execution::allocator_t<ProtoAllocator>` property or the implementation defined default allocator established by the `execution::allocator_t<void>` property.

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

2.5.3.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: `true` if `&a.query(execution::context) == &b.query(execution::context)` and `a` and `b` have identical properties, otherwise `false`.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: `!(a == b)`.

2.5.3.5 static_thread_pool scheduler functions

In addition to conforming to the above specification, `static_thread_pool` schedulers shall conform to the following specification.

```
class C
{
public:
    sender_type schedule() noexcept;
};
```

`c` is a type satisfying the scheduler requirements.

2.5.3.6 Sender creation

```
sender_type schedule() noexcept;
```

Returns: A sender that may be used to submit function objects to the thread pool. The returned sender has the following properties already established:

- `execution::oneway`
- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::outstanding_work.untracked`
- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

2.5.4 `static_thread_pool` sender types

All sender types accessible through `static_thread_pool::scheduler().schedule()`, and subsequent invocations of the member function require, conform to the following specification.

```
class C
{
public:

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // sender operations:

    see-below require(execution::blocking_t::never_t) const;
    see-below require(execution::blocking_t::possibly_t) const;
    see-below require(execution::blocking_t::always_t) const;
    see-below require(execution::relationship_t::continuation_t) const;
    see-below require(execution::relationship_t::fork_t) const;
    see-below require(execution::outstanding_work_t::tracked_t) const;
    see-below require(execution::outstanding_work_t::untracked_t) const;
    see-below require(const execution::allocator_t<void>& a) const;
    template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

    static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t::parallel_t) const;
    static constexpr execution::mapping_t query(execution::mapping_t::thread_t) const;
    execution::blocking_t query(execution::blocking_t) const;
    execution::relationship_t query(execution::relationship_t) const;
    execution::outstanding_work_t query(execution::outstanding_work_t) const;
    see-below query(execution::context_t) const noexcept;
    see-below query(execution::allocator_t<void>) const noexcept;
    template<class ProtoAllocator>
    see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

    bool running_in_this_thread() const noexcept;
};

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
```

Objects of type `c` are associated with a `static_thread_pool`.

2.5.4.1 Constructors

C(const C& other) noexcept;

Postconditions: *this == other.

C(C&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

2.5.4.2 Assignment

C& operator=(const C& other) noexcept;

Postconditions: *this == other.

Returns: *this.

C& operator=(C&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

Returns: *this.

2.5.4.3 Operations

see-below require(execution::blocking_t::never_t) const;
see-below require(execution::blocking_t::possibly_t) const;
see-below require(execution::blocking_t::always_t) const;
see-below require(execution::relationship_t::continuation_t) const;
see-below require(execution::relationship_t::fork_t) const;
see-below require(execution::outstanding_work_t::tracked_t) const;
see-below require(execution::outstanding_work_t::untracked_t) const;

Returns: An sender object of an unspecified type conforming to these specifications, associated with the same thread pool as *this, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned sender object. All other properties of the returned sender object are identical to those of *this.

see-below require(const execution::allocator_t<void>& a) const;

Returns: require(execution::allocator(x)), where x is an implementation-defined default allocator.

```
template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An sender object of an unspecified type conforming to these specifications, associated with the same thread pool as *this, with the execution::allocator_t<ProtoAllocator> property established such that allocation and deallocation associated with function submission will be performed using a copy of a.alloc. All other properties of the returned sender object are identical to those of *this.

static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t) const;

Returns: execution::bulk_guarantee.parallel

static constexpr execution::mapping_t query(execution::mapping_t) const;

Returns: execution::mapping.thread.

execution::blocking_t query(execution::blocking_t) const;
execution::relationship_t query(execution::relationship_t) const;
execution::outstanding_work_t query(execution::outstanding_work_t) const;

Returns: The value of the given property of *this.

static_thread_pool& query(execution::context_t) const noexcept;

Returns: A reference to the associated static_thread_pool object.

see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

Returns: The allocator object associated with the sender, with type and value as either previously established by the execution::allocator_t<ProtoAllocator> property or the implementation defined default allocator established by the execution::allocator_t<void> property.

```
bool running_in_this_thread() const noexcept;
```

Returns: true if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

2.5.4.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: true if `&a.query(execution::context) == &b.query(execution::context)` and `a` and `b` have identical properties, otherwise false.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: `!(a == b)`.

2.5.4.5 `static_thread_pool` sender execution functions

In addition to conforming to the above specification, `static_thread_pool` schedulers' senders shall conform to the following specification.

```
class C
{
public:
    template<template<class...> class Tuple, template<class...> class Variant>
        using value_types = Variant<Tuple<>>>;
    template<template<class...> class Variant>
        using error_types = Variant<>;
    static constexpr bool sends_done = true;

    template<receiver_of R>
        see-below connect(R&& r) const;
};
```

`C` is a type satisfying the `typed_sender` requirements.

```
template<receiver_of R>
    see-below connect(R&& r) const;
```

Returns: An object whose type satisfies the `operation_state` concept.

Effects: When `execution::start` is called on the returned operation state, the receiver `r` is submitted for execution on the `static_thread_pool` according to the the properties established for `*this`. let `e` be an object of type `exception_ptr`; then `static_thread_pool` will evaluate one of `execution::set_value(r)`, `execution::set_error(r, e)`, or `execution::set_done(r)`.

2.5.5 `static_thread_pool` executor types

All executor types accessible through `static_thread_pool::executor()`, and subsequent invocations of the member function `require`, conform to the following specification.

```
class C
{
public:

    // types:

    using shape_type = size_t;
    using index_type = size_t;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // executor operations:

    see-below require(execution::blocking_t::never_t) const;
    see-below require(execution::blocking_t::possibly_t) const;
    see-below require(execution::blocking_t::always_t) const;
    see-below require(execution::relationship_t::continuation_t) const;
    see-below require(execution::relationship_t::fork_t) const;
```



```

see-below require(execution::outstanding_work_t::tracked_t) const;
see-below require(execution::outstanding_work_t::untracked_t) const;
see-below require(const execution::allocator_t<void>& a) const;
template<class ProtoAllocator>
see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t::parallel_t) const;
static constexpr execution::mapping_t query(execution::mapping_t::thread_t) const;
execution::blocking_t query(execution::blocking_t) const;
execution::relationship_t query(execution::relationship_t) const;
execution::outstanding_work_t query(execution::outstanding_work_t) const;
see-below query(execution::context_t) const noexcept;
see-below query(execution::allocator_t<void>) const noexcept;
template<class ProtoAllocator>
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

bool running_in_this_thread() const noexcept;
};

```

```

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;

```

Objects of type `C` are associated with a `static_thread_pool`.

2.5.5.1 Constructors

```
C(const C& other) noexcept;
```

Postconditions: `*this == other`.

```
C(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

2.5.5.2 Assignment

```
C& operator=(const C& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
C& operator=(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

2.5.5.3 Operations

```

see-below require(execution::blocking_t::never_t) const;
see-below require(execution::blocking_t::possibly_t) const;
see-below require(execution::blocking_t::always_t) const;
see-below require(execution::relationship_t::continuation_t) const;
see-below require(execution::relationship_t::fork_t) const;
see-below require(execution::outstanding_work_t::tracked_t) const;
see-below require(execution::outstanding_work_t::untracked_t) const;

```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of `*this`.

```
see-below require(const execution::allocator_t<void>& a) const;
```

Returns: `require(execution::allocator(x))`, where `x` is an implementation-defined default allocator.

```

template<class ProtoAllocator>
see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned executor object are identical to those of `*this`.

```
static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t) const;
```

Returns: execution::bulk_guarantee.parallel

```
static constexpr execution::mapping_t query(execution::mapping_t) const;
```

Returns: execution::mapping.thread.

```
execution::blocking_t query(execution::blocking_t) const;
execution::relationship_t query(execution::relationship_t) const;
execution::outstanding_work_t query(execution::outstanding_work_t) const;
```

Returns: The value of the given property of *this.

```
static_thread_pool& query(execution::context_t) const noexcept;
```

Returns: A reference to the associated static_thread_pool object.

```
see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;
```

Returns: The allocator object associated with the executor, with type and value as either previously established by the execution::allocator_t<ProtoAllocator> property or the implementation defined default allocator established by the execution::allocator_t<void> property.

```
bool running_in_this_thread() const noexcept;
```

Returns: true if the current thread of execution is a thread that was created by or attached to the associated static_thread_pool object.

2.5.5.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: true if &a.query(execution::context) == &b.query(execution::context) and a and b have identical properties, otherwise false.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: !(a == b).

2.5.5.5 static_thread_pool executor execution functions

In addition to conforming to the above specification, static_thread_pool executors shall conform to the following specification.

```
class C
{
public:
    template<class Function>
        void execute(Function&& f) const;

    template<class Function>
        void bulk_execute(Function&& f, size_t n) const;
};
```

C is a type satisfying the Executor requirements.

```
template<class Function>
    void execute(Function&& f) const;
```

Effects: Submits the function f for execution on the static_thread_pool according to the the properties established for *this. If the submitted function f exits via an exception, the static_thread_pool invokes std::terminate().

```
template<class Function>
    void bulk_execute(Function&& f, size_t n) const;
```

Effects: Submits the function f for bulk execution on the static_thread_pool according to properties established for *this. If the submitted function f exits via an exception, the static_thread_pool invokes std::terminate().

2.6 Changelog

2.6.1 Revision 12

Introduced introductory design discussion which replaces the obsolete [P0761](#). No normative changes.

2.6.2 Revision 11

As directed by SG1 at the 2019-07 Cologne meeting, we have implemented the following changes suggested by P1658 and P1660 which incorporate “lazy” execution:

- Eliminated all interface-changing properties.
- Introduced `set_value`, `set_error`, `set_done`, `execute`, `submit`, and `bulk_execute` customization point objects.
- Introduced `executor`, `executor_of`, `receiver`, `receiver_of`, `sender`, `sender_to`, `typed_sender`, and `scheduler` concepts.
- Renamed polymorphic executor to `any_executor`.
- Introduced `invocable_archetype`.
- Eliminated `OneWayExecutor` and `BulkOneWayExecutor` requirements.
- Eliminated `is_executor`, `is_oneway_executor`, and `is_bulk_oneway_executor` type traits.
- Eliminated interface-changing properties from `any_executor`.

2.6.3 Revision 10

As directed by LEWG at the 2018-11 San Diego meeting, we have migrated the property customization mechanism to namespace `std` and moved all of the details of its specification to a separate paper, [P1393](#). This change also included the introduction of a separate customization point for interface-enforcing properties, `require_concept`. The generalization also necessitated the introduction of `is_applicable_property_v` in the properties paper, which in turn led to the introduction of `is_executor_v` to express the applicability of properties in this paper.

2.6.4 Revision 9

As directed by the SG1/LEWG straw poll taken during the 2018 Bellevue executors meeting, we have separated The Unified Executors programming model proposal into two papers. This paper contains material related to one-way execution which the authors hope to standardize with C++20 as suggested by the Bellevue poll. [P1244](#) contains remaining material related to dependent execution. We expect P1244 to evolve as committee consensus builds around a design for dependent execution.

This revision also contains bug fixes to the `allocator_t` property which were originally scheduled for Revision 7 but were inadvertently omitted.

2.6.5 Revision 8

Revision 8 of this proposal makes interface-changing properties such as `oneway` mutually exclusive in order to simplify implementation requirements for executor adaptors such as polymorphic executors. Additionally, this revision clarifies wording regarding execution agent lifetime.

2.6.6 Revision 7

Revision 7 of this proposal corrects wording bugs discovered by the authors after Revision 6’s publication.

- Enhanced `static_query_v` to result in a default property value for executors which do not provide a `query` function for the property of interest
- Revise `then_execute` and `bulk_then_execute`’s operational semantics to allow user functions to handle incoming exceptions thrown by preceding execution agents
- Introduce `exception_arg` to disambiguate the user function’s exceptional overload from its nonexceptional overload in `then_execute` and `bulk_then_execute`

2.6.7 Revision 6

Revision 6 of this proposal corrects bugs and omissions discovered by the authors after Revision 5’s publication, and introduces an enhancement improving the safety of the design.

- Enforce mutual exclusion of behavioral properties via the type system instead of via convention
- Introduce missing `execution::require` adaptations
- Allow executors to opt-out of invoking factory functions when appropriate
- Various bug fixes and corrections

2.6.8 Revision 5

Revision 5 of this proposal responds to feedback requested during the 2017 Albuquerque ISO C++ Standards Committee meeting

and introduces changes which allow properties to better interoperate with polymorphic executor wrappers and also simplify `execution::require`'s behavior.

- Defined general property type requirements
- Elaborated specification of standard property types
- Simplified `execution::require`'s specification
- Enhanced polymorphic executor wrapper
 - Templatized `execution::executor<SupportableProperties...>`
 - Introduced `prefer_only` property adaptor
- Responded to Albuquerque feedback
 - From SG1
 - Execution contexts are now optional properties of executors
 - Eliminated ill-specified caller-agent forward progress properties
 - Elaborated `Future`'s requirements to incorporate forward progress
 - Reworded operational semantics of execution functions to use similar language as the blocking properties
 - Elaborated `static_thread_pool`'s specification to guarantee that threads in the pool boost-block their work
 - Elaborated operational semantics of execution functions to note that forward progress guarantees are specific to the concrete executor type
 - From LEWG
 - Eliminated named `BaseExecutor` concept
 - Simplified general executor requirements
 - Enhanced the `OneWayExecutor` introductory paragraph
 - Eliminated `has_*_member` type traits
- Minor changes
 - Renamed TS namespace from `concurrency_v2` to `executors_v1`
 - Introduced `static_query_v` enabling static queries
 - Eliminated unused `property_value` trait
 - Eliminated the names `allocator_wrapper_t` and `default_allocator`

2.6.9 Revision 4

- Specified the guarantees implied by `bulk_sequenced_execution`, `bulk_parallel_execution`, and `bulk_unsequenced_execution`

2.6.10 Revision 3

- Introduced `execution::query()` for executor property introspection
- Simplified the design of `execution::prefer()`
- `oneway`, `twoway`, `single`, and `bulk` are now `require()`-only properties
- Introduced properties allowing executors to opt into adaptations that add blocking semantics
- Introduced properties describing the forward progress relationship between caller and agents
- Various minor improvements to existing functionality based on prototyping

2.6.11 Revision 2

- Separated wording from explanatory prose, now contained in paper [P0761](#)
- Applied the simplification proposed by paper [P0688](#)

2.6.12 Revision 1

- Executor category simplification
- Specified executor customization points in detail
- Introduced new fine-grained executor type traits
 - Detectors for execution functions
 - Traits for introspecting cross-cutting concerns
 - Introspection of mapping of agents to threads
 - Introspection of execution function blocking behavior
- Allocator support for single agent execution functions
- Renamed `thread_pool` to `static_thread_pool`
- New introduction

2.6.13 Revision 0

- Initial design

2.7 Appendix: Executors Bibilography

Paper	Notes	Date introduced
N3378 - A preliminary proposal for work executors		
N3562 - Executors and schedulers, revision 1		
N3731 - Executors and schedulers, revision 2		
N3785 - Executors and schedulers, revision 3	Initial executors proposal from Google, based on an abstract base class.	2012-02-24
N4143 - Executors and schedulers, revision 4		
N4414 - Executors and schedulers, revision 5		
P0008 - C++ Executors		
N4046 - Executors and Asynchronous Operations	Initial executors proposal from Kohlhoff, based on extensions to ASIO.	2014-05-26
N4406 - Parallel Algorithms Need Executors		
P0058 - An interface for abstracting execution	Initial executors proposal from Nvidia, based on a traits class.	2015-04-10
P0285 - Using customization points to unify executors	Proposes unifying various competing executors proposals via customization points.	2016-02-14
P0443 - A Unified Executors Proposal for C++	This proposal.	2016-10-17
P0688 - A Proposal to Simplify the Executors Design	Proposes simplifying this proposal's APIs using properties.	2017-06-19
P0761 - Executors Design Document	Describes the design of this proposal circa 2017.	2017-07-31
P1055 - A Modest Executor Proposal	Initial executors proposal from Facebook, based on lazy execution.	2018-04-26
P1194 - The Compromise Executors Proposal: A lazy simplification of P0443	Initial proposal to integrate senders and receivers into this proposal.	2018-10-08
P1232 - Integrating executors with the standard library through customization	Proposes to allow executors to customize standard algorithms directly.	2018-10-08
P1244 - Dependent Execution for a Unified Executors Proposal for C++	Vestigial futures-based dependent execution functionality excised from later revisions of this proposal.	2018-10-08
P1341 - Unifying asynchronous APIs in C++ standard Library	Proposes enhancements making senders awaitable.	2018-11-25
P1393 - A General Property Customization Mechanism	Standalone paper proposing the property customization used by P0443 executors.	2019-01-13
P1677 - Cancellation is serendipitous-success	Motivates the need for done in addition to error.	2019-05-18
P1678 - Callbacks and Composition	Argues for callbacks/receivers as a universal design pattern in the standard library.	2019-05-18
P1525 - One-Way execute is a Poor Basis Operation	Identifies deficiencies of execute as a basis operation.	2019-06-17
P1658 - Suggestions for Consensus on Executors	Suggests progress-making changes to this proposal circa 2019.	2019-06-17
P1660 - A Compromise Executor Design Sketch	Proposes concrete changes to this proposal along the lines of P1525 , P1658 , and P1738 .	2019-06-17
P1738 - The Executor Concept Hierarchy Needs a Single Root	Identifies problems caused by a multi-root executor concept hierarchy.	2019-06-17
P1897 - Towards C++23 executors: A proposal for an initial set of algorithms	Initial proposal for a set of customizable sender algorithms.	2019-10-06
P1898 - Forward progress delegation for executors	Proposes a model of forward progress for executors and asynchronous graphs of work.	2019-10-06

[P2006 - Splitting submit\(\) into connect\(\)/start\(\)](#)

Proposes refactoring `submit` into more fundamental `connect` and `start` sender operations.

2020-01-13

[P2033 - History of Executor Properties](#)

Documents the evolution of [P1393](#)'s property system, especially as it relates to executors.

2020-01-13

2.8 Appendix: A note on coroutines

[P1341](#) leverages the structural similarities between coroutines and the sender/receiver abstraction to give a class of senders a standard-provided operator `co_await`. The end result is that a sender, simply by dint of being a sender, can be `co_await`-ed in a coroutine. With the refinement of sender/receiver that was proposed in [P2006](#) — namely, the splitting of `submit` into `connect/start` — that automatic adaptation from sender-to-awaitable is allocation- and synchronization-free.

2.9 Appendix: The retry Algorithm

Below is an implementation of a simple `retry` algorithm in terms of `sender/receiver`. This algorithm is Generic in the sense that it will retry any multi-shot asynchronous operation that satisfies the `sender` concept. More accurately, it takes any deferred `async` operation and wraps it so that when it is executed, it will retry the wrapped operation until it either succeeds or is cancelled.

Full working code can be found here: <https://godbolt.org/z/nm6GmH>

```
// _conv needed so we can emplace construct non-movable types into
// a std::optional.
template<invocable F>
    requires std::is_nothrow_move_constructible_v<F>
struct _conv {
    F f_;
    explicit _conv(F f) noexcept : f_((F&&) f) {}
    operator invoke_result_t<F>() && {
        return ((F&&) f_());
    }
};

// pass through set_value and set_error, but retry the operation
// from set_error.
template<class O, class R>
struct _retry_receiver {
    O* o_;
    explicit _retry_receiver(O* o): o_(o) {}
    template<class... As>
        requires receiver_of<R, As...>
    void set_value(As&&... as) &&
        noexcept(is_nothrow_receiver_of_v<R, As...>) {
        ::set_value(std::move(o_->r_), (As&&) as...);
    }
    void set_error(auto&&) && noexcept {
        o_->retry(); // This causes the op to be retried
    }
    void set_done() && noexcept {
        ::set_done(std::move(o_->r_));
    }
};

template<sender S>
struct _retry_sender : sender_base {
    S s_;
    explicit _retry_sender(S s): s_((S&&) s) {}

    // Hold the nested operation state in an optional so we can
    // re-construct and re-start it when the operation fails.
    template<receiver R>
    struct _op {
        S s_;
        R r_;
        std::optional<state_t<S&, _retry_receiver<_op, R>>> o_;

        _op(S s, R r): s_((S&&)s), r_((R&&)r), o_{_connect()} {}
        _op(_op&&) = delete;

        auto _connect() noexcept {
            return _conv{[this] {
                return ::connect(s_, _retry_receiver<_op, R>{this});
            }};
        }
    };
};
```

```

void _retry() noexcept try {
    o_.emplace(_connect()); // potentially throwing
    ::start(std::move(*o_));
} catch(...) {
    ::set_error((R&&) r_, std::current_exception());
}
void start() && noexcept {
    ::start(std::move(*o_));
}
};

template<receiver R>
    requires sender_to<S&, _retry_receiver<_op<R>, R>>
auto connect(R r) && -> _op<R> {
    return _op<R>{(S&&) s_, (R&&) r};
}
};

template<sender S>
sender auto retry(S s) {
    return _retry_sender{(S&&)s};
}

```